



Project Report On

Fog Scheduler using Quantum Principles

*Submitted in partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology

in

Computer Science & Engineering

By

Adithyan Ajayan (U2203018)

Ajay Mukund A. (U2203020)

Alex Babu (U2203030)

Anna George (U2203049)

Bennette A. Sabu (U2203066)

Under the guidance of

Dr. Jisha G.

**Department of Computer Science & Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)**

Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

*This is to certify that the project report entitled "**Fog Scheduler using Quantum Principles**" is a bonafide record of the work done by **Adithyan Ajayan (U2203018)**, **Ajay Mukund A. (U2203020)**, **Alex Babu (U2203030)**, **Anna George (U2203049)** and **Bennette A. Sabu (U2203066)**, submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in "Computer Science & Engineering" during the academic year 2025-2026.*

Project Guide

Dr. Jisha G.
Associate Professor
Dept. of CSE
RSET

Project Coordinator

Dr. Jisha G.
Associate Professor
Dept. of CSE
RSET

Head of Department

Dr. Preetha K.G.
Professor
Dept. of CSE
RSET

ACKNOWLEDGMENT

I wish to express my sincere gratitude towards **Rev. Fr. Dr. Jaison Paul Mullerikkal**, Principal of RSET, and **Dr. Preetha K. G.**, Head of the Department of "Computer Science & Engineering" for providing me with the opportunity to undertake my project, "Fog Scheduler using Quantum Principles".

I am highly indebted to my project coordinator, **Dr. Jisha G.**, Associate Professor, Department of Computer Science and Engineering, and the faculty-in-charge, **Dr. Preetha K. G.**, Head of the Department of Computer Science & Engineering for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Dr. Jisha G.**, Associate Professor, Department of Computer Science and Engineering, for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Adithyan Ajayan

Ajay Mukund A.

Alex Babu

Anna George

Bennette A. Sabu

Abstract

The rapid expansion of Internet of Things (IoT) devices has shifted the computing paradigm toward fog computing environments, where processing occurs closer to the data source to meet low-latency requirements. However, traditional scheduling mechanisms often struggle to navigate the complex interactions of heterogeneous resources and dynamic network conditions, leading to inefficient resource utilization and high energy consumption. This problem is further amplified in vehicular networks where rapid mobility causes constant fluctuations in network topology and service availability.

To address these challenges, this project proposes a Quantum-Inspired Genetic Algorithm (QIGA) that leverages qubit representation and adaptive rotation gates to enhance solution diversity and convergence speed. The research was conducted in two phases: Phase 1 involved building a simulation framework to benchmark the QIGA against classical metaheuristics like GA and PSO in a 3-tier (Edge-Fog-Cloud) architecture. Phase 2 extended this framework by integrating the SUMO traffic simulator with EdgeSimPy to create a dynamic vehicular orchestration loop. Extensive simulations demonstrate that the proposed method achieves an average reduction in energy consumption of 15.7% and improves latency by up to 69.9%. These results highlight the system's effectiveness in maintaining high Quality of Service (QoS) and meeting task deadlines in highly mobile and resource-constrained environments.

Contents

Acknowledgment	i
Abstract	ii
List of Abbreviations	vi
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background	1
1.2 Problem Definition	1
1.3 Scope and Motivation	1
1.4 Objectives	2
1.4.1 PHASE 1	2
1.4.2 PHASE 2	2
1.5 Challenges	3
1.6 Assumptions	3
1.7 Societal / Industrial Relevance	4
1.8 Organization of the Report	4
1.9 Summary of Chapter	4
2 Literature Survey	5
2.1 Qadri et al. (2024). Optimizing Traffic Signal Timing at Urban Intersections: A Simheuristic Approach Using GA and SUMO. <i>IEEE Transactions on Intelligent Transportation Systems</i>	5
2.2 Wu et al. (2025). Challenges and Opportunities in Fog Computing Scheduling: A Literature Review. <i>IEEE Access</i>	6

2.3	Ahanger et al. (2023). Quantum-Inspired Task Optimization algorithm specifically for IoT Edge-Fog computing environments. <i>Future Generation Computer Systems</i>	7
2.4	Madasamy et al. (2024). Predictive EQCi-Optimized Load Scheduling for Heterogeneous IoT-Data in Fog Computing Environments. <i>IEEE Internet of Things Journal</i>	8
2.5	Raj & Edwin (2025). Real-Time Vehicle Trajectory and State Prediction using Advanced Recurrent Networks in a SUMO Simulated Environment. <i>Expert Systems with Applications</i>	9
2.6	Comparison of Reviewed Literature	9
2.7	Research Gaps Identified	10
2.8	Summary of Chapter	11
3	System Design	12
3.1	Overall Architecture	12
3.1.1	System Workflow Overview	12
3.1.2	Phase 1: Fog Scheduler Static System	13
3.1.3	Phase 2: Vehicular Fog Orchestration	14
3.2	Component Design	16
3.2.1	Behavioral Design: Sequence Diagrams	17
3.3	Module Division	18
3.4	Tools and Technologies	19
3.5	Work Breakdown and Responsibilities	20
3.6	Key Deliverables	21
3.7	Project Timeline	22
3.8	Summary of Chapter	23
4	Proposed Methodology: SHybridQIGA	24
4.1	System Architecture and Problem Formulation	24
4.1.1	Task and Resource Modeling	24
4.1.2	Delay Calculation (Multi-Core Physics Engine)	25
4.1.3	Multi-Objective Fitness Function	25
4.2	Proposed Algorithm: SHybridQIGA	26

4.2.1	Quantum Chromosome Representation	26
4.2.2	Heuristic Seeding Initialization	26
4.2.3	Quantum Measurement and Evolutionary Rotation	26
4.3	The Multi-Core Load-Aware Repair Mechanism	27
4.4	Summary of the Algorithmic Workflow	28
5	Results and Discussions	30
5.1	Phase 1: Static Heuristic Evaluation	30
5.1.1	Latency vs. Energy Trade-offs	30
5.1.2	Limitations under Network Saturation	34
5.2	Phase 2: Dynamic Vehicular Edge Evaluation	37
5.2.1	Standardized Traffic Injection Profile	37
5.2.2	Scenario 1: Baseline Urban Grid	37
5.2.3	Scenario 2: Edge-Heavy Infrastructure	39
5.2.4	Scenario 3: Server-Heavy Infrastructure	40
5.2.5	Scenario 4: Ultra-Dense Infrastructure	41
5.2.6	Cross-Scenario Algorithmic Footprint Analysis	41
5.3	Chapter Summary	43
6	Conclusions & Future Scope	44
6.1	Conclusion	44
6.2	Future Scope	44
	References	46
	Appendix A: Presentation	48
	Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes	74
	Appendix C: CO-PO-PSO Mapping	79

List of Abbreviations

API	- Application Programming Interface
BiLSTM	- Bidirectional Long Short-Term Memory
DE	- Differential Evolution
GA	- Genetic Algorithm
GRU	- Gated Recurrent Unit
IoT	- Internet of Things
LSP	- Load Scheduling Policy
MOHEFT	- Multi-Objective Heterogeneous Earliest Finish Time
NAI	- Node Availability Index
OC	- Only Cloud
OE	- Opportunistic Edge
PSO	- Particle Swarm Optimization
Q-bit	- Quantum bit
QCi	- Quantum Computing-inspired
QIGA	- Quantum-Inspired Genetic Algorithm
QoS	- Quality of Service
RA	- Random Allocation
RNN	- Recurrent Neural Network
RR	- Round Robin
RSU	- Roadside Unit
RT	- Real-Time
SHybridQIGA	- Smart Hybrid Quantum-Inspired Genetic Algorithm
SLA	- Service Level Agreement
SQIGA	- Standard Quantum-Inspired Genetic Algorithm
SUMO	- Simulation of Urban MObility
TraCI	- Traffic Control Interface
VEC	- Vehicular Edge Computing

List of Figures

3.1	Phase 1 Architecture Diagram	14
3.2	Phase 2 Architecture Diagram	16
3.3	Phase 1 Sequence Diagram	17
3.4	Phase 2 Sequence Diagram	18
3.5	Gantt Chart	22
5.1	Average energy consumption in the compute-heavy scenario	33
5.2	Performance comparison in the balanced scenario	34
5.3	Weighted fitness scores in the high-traffic scenario	35
5.4	Standardized vehicular traffic injection profile	38
5.5	SLA violation rate over time (Baseline Scenario)	38
5.6	Total energy drain over time (Baseline Scenario)	38
5.7	Pareto frontier of energy vs. SLA violations (Server-Heavy Scenario)	40
5.8	Multi-objective radar chart comparison across all infrastructure scenarios .	42

List of Tables

2.1	Comparison of Reviewed Literature on Scheduling and Mobility Frameworks	10
3.1	Work Breakdown and Responsibilities	20
5.1	Performance Comparison in Critical Real-Time Scenario	31
5.2	Performance Comparison in Compute-Heavy Scenario	32
5.3	Performance Comparison in Balanced Scenario	33
5.4	Performance Comparison in Heavy Data Scenario	35
5.5	Performance Comparison in High Traffic Scenario	36
5.6	Cumulative Performance in the Baseline Urban Grid	38
5.7	Cumulative Performance in the Edge-High Infrastructure	39
5.8	Cumulative Performance in the Server-High Infrastructure	40
5.9	Cumulative Performance in the Increased Infrastructure Scenario	41

Chapter 1

Introduction

1.1 Background

The rapid expansion of Internet of Things (IoT) devices and low-latency applications has shifted the computing paradigm from centralized clouds to the network edge. Current scenarios in Mobile Edge Computing (MEC) and Vehicular Fog Computing (VFC) involve heterogeneous resources where task scheduling is critical for optimizing performance [1]. Realizing the full potential of these environments is challenging due to the dynamic nature of network conditions and varying computational capacities of edge nodes. Efficient orchestration is vital to reducing energy consumption and latency for end-users in urban and mobile settings. This project is important as it addresses these complexities by integrating advanced quantum-inspired algorithms with realistic traffic and network simulations [2][3].

1.2 Problem Definition

The project aims to develop and validate a high-performance orchestration framework for vehicular fog environments that optimizes task allocation across a multi-tier architecture. The core problem is defined as the efficient scheduling of heterogeneous, time-sensitive tasks to maximize resource utilization while minimizing energy consumption and latency in highly dynamic vehicular networks [4][5].

1.3 Scope and Motivation

The project scope involves developing a 3-tier Edge-Fog-Cloud architecture and a benchmarking suite of nine scheduling algorithms, including quantum-inspired and classical metaheuristics. It utilizes SUMO and EdgeSimPy to simulate dynamic vehicular mobil-

ity and network fluctuations across diverse urban and open-area scenarios. Performance is evaluated based on conflicting metrics such as makespan, energy efficiency, and resource utilization.

The motivation for this work is driven by the inability of traditional scheduling methods to efficiently handle the high-dimensional search spaces and uncertainty of heterogeneous vehicular networks. By applying quantum computing principles like qubit representation, the project aims to significantly improve convergence speed and solution diversity over standard evolutionary algorithms [6][7]. Ultimately, this research seeks to provide a scalable and robust orchestration framework capable of sustaining high Quality of Service for latency-sensitive smart city applications.

1.4 Objectives

1.4.1 PHASE 1

To design a robust simulation environment that optimizes task offloading in a heterogeneous 3-Tier network.

- **Develop a 3 – Tier Network:** Create a high-fidelity virtual model of Edge, Fog, and Cloud layers to simulate realistic network behavior.
- **Multi-Objective Optimization:** To develop an algorithm which simultaneously minimize Latency (response time), Energy Consumption (power usage), and Monetary Cost (cloud fees).
- **Algorithm Benchmarking:** Compare Quantum-Inspired algorithms (QIGA, HybridQIGA) against standard evolutionary methods (GA, PSO, NSGA-II) to prove efficiency gains.
- **Heterogeneous Environment:** Evaluate performance under resource scarcity by modeling specific hardware constraints (e.g., Raspberry Pi vs. Cloud Servers).

1.4.2 PHASE 2

To apply the scheduling logic to a dynamic vehicular environment where continuous physical mobility and diverse data applications disrupt the network state.

- **Cyber-Physical Integration:** Integrate the SUMO traffic simulator via the TraCI protocol to transition from a stationary topology into a continuous, physics-based smart city environment.
- **Spatial-to-Network Translation:** Map SUMO coordinates (X/Y) onto the EdgeSimPy grid to track moving vehicles and trigger dynamic network handovers as cars switch base stations.
- **Heterogeneous Workload Emulation:** Treat vehicles as independent nodes generating diverse tasks (telemetry pings, vision analytics) to simulate network bursts during physical traffic clustering.
- **Temporal Resilience Validation:** Execute the algorithm in a continuous time-series loop to monitor recovery speeds and minimize SLA violations during unpredictable traffic spikes.

1.5 Challenges

The project faces the challenge of managing the high degree of uncertainty caused by rapid fluctuations in network bandwidth and node availability in vehicular environments. Coordinating task execution across heterogeneous hardware with varying power profiles and computational limits adds significant algorithmic complexity. Additionally, achieving fast convergence in multi-objective optimization without sacrificing solution diversity remains a primary technical hurdle.

1.6 Assumptions

- The system assumes a set of m edge servers and n fog nodes with known, static computational and storage capacities for the duration of a single simulation step.
- It is assumed that task workloads, data sizes, and deadlines are pre-defined or can be estimated with a small degree of injected prediction noise (e.g., $\pm 10\%$).
- Network link characteristics like bandwidth and latency are assumed to be time-dependent functions of the physical distance between nodes as determined by the mobility models.

1.7 Societal / Industrial Relevance

This project is highly relevant to the development of Intelligent Transportation Systems (ITS) and smart city infrastructure, where real-time data processing is essential for traffic management and autonomous driving safety. Industrially, the framework can be applied by telecommunications providers and edge computing vendors to optimize resource allocation, thereby reducing operational costs and energy overhead in large-scale distributed networks.

1.8 Organization of the Report

This report is organized into six chapters to systematically present the research and development of the proposed fog scheduling framework. Chapter 1 introduces the background, problem definition, objectives, and overall motivation of the project. Chapter 2 reviews literature on fog computing scheduling, vehicular edge computing, and quantum-inspired optimization techniques, along with the identification of key research gaps. Chapter 3 describes the system design and architectural framework, detailing the two-phase development approach, modules, and tools used for implementation. Chapter 4 explains the proposed methodology, introducing the Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA), its mathematical formulation, fitness function, and load-aware repair mechanism. Chapter 5 presents the experimental results and discussions, including performance evaluation across multiple scenarios and comparisons with existing algorithms. Finally, Chapter 6 summarizes key findings and suggests future research directions.

1.9 Summary of Chapter

This chapter established the foundational framework for vehicular fog orchestration, highlighting the transition from traditional cloud models to dynamic, quantum-optimized edge environments. It detailed the implementation of the QIGA-based scheduler and its integration with vehicular mobility simulators to address the limitations of existing metaheuristics. The proposed multi-objective approach provides a scalable solution for managing the complex interplay of latency, energy, and resource use in future mobile networks.

Chapter 2

Literature Survey

The paradigm shift from centralized cloud computing to distributed Vehicular Edge Computing (VEC) has introduced significant challenges in dynamic resource allocation, energy management, and strict latency adherence. To contextualize the contributions of this thesis, it is imperative to analyze the existing body of knowledge regarding scheduling algorithms and simulation frameworks. This chapter reviews recent literature, focusing on recent advancements in Vehicular Mobility Frameworks, Traditional Meta-Heuristic Scheduling, and Quantum-Inspired Scheduling Models. These works provide insights into network architectures, data optimization techniques, and lightweight mathematical models that enhance computational efficiency.

2.1 Qadri et al. (2024). Optimizing Traffic Signal Timing at Urban Intersections: A Simheuristic Approach Using GA and SUMO. *IEEE Transactions on Intelligent Transportation Systems*.

This paper explores a simheuristic framework integrating the Simulation of Urban Mobility (SUMO) with a Genetic Algorithm (GA) to optimize traffic signal timing at urban intersections. The study demonstrates the efficacy of using the Traffic Control Interface (TraCI) to extract real-time vehicle dynamics and inject them into a meta-heuristic optimization loop to minimize traffic congestion. While their focus was on traffic light phases rather than computational offloading, the study established the viability of SUMO as a robust digital twin for heuristic validation.

Strengths

- Successfully links meta-heuristic algorithms with dynamic physical traffic via the TraCI API.

- Provides a highly realistic microscopic simulation environment for testing algorithmic responsiveness.
- Demonstrates significant reductions in urban traffic bottlenecks.

Limitations

- Focused strictly on traffic signal phases; does not address computational edge of-loading or server energy limits.
- The Genetic Algorithm used struggles with rapid convergence speeds during sudden traffic spikes.

Relevance to Project: This work is highly relevant as it establishes the foundational methodology for using SUMO and TraCI. However, unlike this study, our project utilizes SUMO not to control traffic lights, but to generate dynamic computational workloads for edge servers.

2.2 Wu et al. (2025). Challenges and Opportunities in Fog Computing Scheduling: A Literature Review. *IEEE Access*.

This paper provides a comprehensive literature review on the challenges and opportunities in fog computing scheduling. It highlights that while Internet of Things (IoT) applications face severe constraints regarding computing availability and energy consumption, centralized cloud computing exacerbates these issues through high communication latency. The survey underscores the growing reliance on meta-heuristic algorithms (e.g., GA, Particle Swarm Optimization) to balance these multi-objective trade-offs.

Strengths

- Provides a clear, comprehensive taxonomy of fog computing constraints including energy, security, and latency.
- Critically analyzes the shortcomings of relying purely on centralized cloud computing for real-time tasks.

Limitations

- As a literature review, it identifies heuristic shortcomings but does not propose a novel architectural solution.
- Does not address the specific physical physics of multi-core server queueing in dense environments.

Relevance to Project: This paper validates the core problem our project aims to solve: the inadequacy of cloud computing for low-latency tasks. It justifies our decision to deploy a localized, multi-tier Edge/Fog architecture [8].

2.3 Ahanger et al. (2023). Quantum-Inspired Task Optimization algorithm specifically for IoT Edge-Fog computing environments. *Future Generation Computer Systems*.

Ahanger et al. propose a Quantum-Inspired Task Optimization algorithm to address the slow convergence rates of traditional evolutionary algorithms. Their model utilizes quantum bits (qubits) to represent scheduling solutions, allowing the algorithm to evaluate vast solution spaces simultaneously. The study demonstrates significant improvements in execution time and energy efficiency compared to standard classical algorithms like standard GA.

Strengths

- Utilizes quantum superposition to achieve faster convergence and better energy efficiency than classical GA.
- Effectively handles complex, multi-objective search spaces inherent in edge computing.

Limitations

- Evaluated primarily in static mathematical environments rather than continuous physical simulations.
- Ignores dynamic vehicular mobility and sudden burstiness of traffic jams.

Relevance to Project: This work is highly usable for our project since our algorithm also employs Quantum-Inspired principles (Qubits and Rotation Gates) to speed up task

mapping. However, our model extends this by introducing a multi-core load-aware repair mechanism [9].

2.4 Madasamy et al. (2024). Predictive EQCi-Optimized Load Scheduling for Heterogeneous IoT-Data in Fog Computing Environments. *IEEE Internet of Things Journal*.

This paper presents a predictive EQCi-optimized load scheduling technique tailored for heterogeneous IoT data. By representing Load Scheduling Policies (LSPs) as qubits, their model dynamically evaluates probabilistic Execution Index values and Node Availability Indices (NAI). Validated using the iFogSim toolkit, their predictive model successfully minimizes execution delays and enhances throughput by synergizing quantum representation with predictive neural network allocation.

Strengths

- Strong predictive capability using a Node Availability Index to anticipate the best server for a task.
- Effectively minimizes execution delays in simulated fog computing environments.
- Validated rigorously against existing load scheduling methods using iFogSim.

Limitations

- Assumes constant server processing speeds, lacking realistic multi-core physical simulation scaling.
- High computational overhead due to the integration of a Neural Network inside the scheduling loop.

Relevance to Project: This concept is important for understanding how quantum bits can represent scheduling policies. However, unlike this study which relies on a heavy neural network for prediction, our project utilizes a lightweight deterministic repair mechanism to ensure sub-second responsiveness on edge nodes.

2.5 Raj & Edwin (2025). Real-Time Vehicle Trajectory and State Prediction using Advanced Recurrent Networks in a SUMO Simulated Environment. *Expert Systems with Applications*.

This study utilizes SUMO to extract high-quality, time-series mobility data for real-time vehicle trajectory and state prediction. By deploying advanced Recurrent Neural Networks (RNNs), including Bidirectional Long Short-Term Memory (BiLSTM) and Gated Recurrent Units (GRU), they demonstrate how SUMO’s simulated environment can train highly accurate predictive models for autonomous obstacle avoidance and state tracking.

Strengths

- Achieves exceptionally high accuracy in real-time vehicular trajectory and state prediction.
- Proves the viability of extracting rich, time-series spatial data directly from SUMO.

Limitations

- Focuses entirely on vehicle movement prediction and lacks a scheduling or resource allocation mechanism.
- Does not address the computational offloading burden that autonomous trajectory tracking places on the network.

Relevance to Project: This work confirms that SUMO provides the necessary spatial-temporal dynamics required for rigorous algorithmic testing. It validates our use of SUMO as the foundational traffic injector for generating dynamic computational tasks based on vehicle movement.

2.6 Comparison of Reviewed Literature

Table 2.1 presents a summarized comparison of the key methodologies discussed in the literature, highlighting their respective advantages and limitations.

Table 2.1: Comparison of Reviewed Literature on Scheduling and Mobility Frameworks

Paper Title	Approach / Methodology	Strengths	Limitations
Optimizing Traffic Signal Timing at Urban Intersections: A Simheuristic Approach Using GA and SUMO [2]	GA integrated with SUMO	Effectively links meta-heuristics with dynamic physical traffic via TraCI.	Focused strictly on traffic signals; does not address computational edge offloading.
Challenges and Opportunities in Fog Computing Scheduling: A Literature Review [4]	Extensive Literature Review	Provides a clear taxonomy of fog constraints (energy, security, latency).	Identifies heuristic shortcomings but does not propose a novel architectural solution.
Quantum-Inspired Task Optimization algorithm specifically for IoT Edge-Fog computing environments [6]	Quantum-Inspired Task Optimization	Faster convergence and better energy efficiency than classical GA.	Evaluated in static environments; ignores dynamic vehicular mobility and queuing bottlenecks.
Predictive EQCi-Optimized Load Scheduling for Heterogeneous IoT-Data in Fog Computing Environments [10]	Predictive EQCi-Optimized Scheduling	Strong predictive capability using Node Availability Index; validated in iFogSim.	Assumes constant server processing speeds; lacks multi-core physical simulation scaling.
Real-Time Vehicle Trajectory and State Prediction using Advanced Recurrent Networks in a SUMO Simulated Environment [11]	RNNs (GRU/BiLSTM) with SUMO	High accuracy in real-time trajectory and state prediction.	Lacks a scheduling or resource allocation mechanism for edge nodes.

2.7 Research Gaps Identified

From the reviewed literature, the following gaps have been identified:

- **Lack of Multi-Core Load Awareness:** Most existing meta-heuristic and quantum models rely on simplistic, static mathematical formulas to calculate execution delays. They fail to account for the physical reality of multi-core CPU queuing.

When a server is overloaded, its effective frequency drops, leading to catastrophic task failures during traffic spikes.

- **Over-reliance on "Best-Effort" Routing:** Traditional algorithms prioritize minimizing theoretical energy functions. Consequently, they tend to blindly dump computational tasks onto low-power nodes, leading to massive queue congestion and SLA violations that the algorithms cannot dynamically repair.
- **Absence of Surgical Constraint Repair:** While quantum models provide excellent global search speeds, they lack deterministic constraint satisfaction. When a generated quantum chromosome violates a strict vehicular deadline, existing systems either discard the solution or assign a static penalty, rather than intelligently mutating and repairing the specific failing task gene.
- **Disconnect Between Static Heuristics and Physical Mobility:** Many quantum scheduling algorithms are validated in static mathematical simulators (like iFogSim). There is a distinct lack of frameworks that seamlessly integrate quantum-inspired meta-heuristics with continuous, dynamic vehicular simulators like SUMO to test true physical transmission and handover delays.

These gaps justify the need for an interactive, dynamic digital twin environment that integrates a lightweight, real-time evaluation model (SHybridQIGA) capable of performing surgical, multi-core load-aware repairs.

2.8 Summary of Chapter

This literature review identifies the current state of Vehicular Edge Computing (VEC), noting that while mobility frameworks (SUMO) and meta-heuristic or Quantum-Inspired algorithms excel in theory, they struggle with real-time volatility. Specifically, existing models fail to address multi-core CPU bottlenecks, dynamic SLA repair, and the integration of recognition technology with continuous mobility. These gaps necessitate a tailored approach—bridging quantum search speeds with deterministic physical constraints—which serves as the foundation for the SHybridQIGA model detailed in the following chapters.

Chapter 3

System Design

This chapter details the architectural framework and design principles of the vehicular fog orchestration system. To address the complexities of task scheduling in both stationary and mobile environments, the system development is divided into two distinct phases. Phase 1 establishes the core "Digital Twin" simulation and benchmarking suite, focusing on the comparison of various metaheuristic algorithms within a 3-tier network. Phase 2 extends this framework into a mobility-aware environment by integrating the SUMO traffic simulator. Vehicular mobility data obtained through the TraCI interface is used to dynamically generate workloads and update network topology within the EdgeSimPy simulation environment.

3.1 Overall Architecture

3.1.1 System Workflow Overview

The operational workflow of the proposed fog orchestration framework is structured in two progressive phases. The system is first evaluated within a controlled static fog environment to benchmark scheduling algorithms, and is then extended to a mobility-aware vehicular fog scenario where dynamic workloads are generated from moving vehicles.

1. In Phase 1, a static Edge–Fog–Cloud infrastructure is constructed within the EdgeSimPy simulation environment to evaluate the performance of multiple scheduling algorithms under controlled network conditions.
2. The orchestration engine executes the SHybridQIGA scheduling algorithm along with baseline algorithms to determine optimal task-to-node assignments across Edge, Fog, and Cloud resources.

3. In Phase 2, vehicular mobility is generated within the SUMO simulator using pre-defined traffic routes and mobility models.
4. Real-time vehicle coordinates and mobility events are retrieved through the TraCI interface and supplied to the EdgeSimPy simulation environment.
5. Based on the vehicular state and dynamic network topology, computational tasks are generated and scheduled across available fog nodes using the SHybridQIGA algorithm.
6. The selected computing nodes execute the assigned tasks while system performance metrics such as latency, energy consumption, and cost are continuously monitored through the Digital Twin dashboard.

This two-phase workflow enables the system to first benchmark scheduling strategies in a controlled static environment and subsequently evaluate their performance under realistic mobility-driven workloads in a vehicular fog computing scenario.

3.1.2 Phase 1: Fog Scheduler Static System

The Phase 1 architecture is designed as a controlled benchmarking environment to evaluate algorithm efficiency.

- **3-Tier Network Model:** The system organizes resources into a hierarchy consisting of Edge Nodes (Tier-1), Fog Servers (Tier-2), and the Cloud (Tier-3).
- **Orchestration Engine:** This central component houses the implementation of nine scheduling algorithms, including QIGA, MOHEFT, and various metaheuristic benchmarks.
- **Digital Twin Dashboard:** A Streamlit-based interface serves as the monitoring layer, providing real-time physics-based metrics such as energy consumption and latency delays.
- **Physics Engine:** Unlike a standard simulator, this layer injects prediction noise and calculates energy using a $Power \times Time$ model to mimic real-world hardware constraints.

The architectural structure of the static benchmarking environment is illustrated in Figure 3.1. The diagram highlights the hierarchical organization of the Edge–Fog–Cloud infrastructure and the interaction between the orchestration engine, the physics engine, and the Digital Twin monitoring interface.

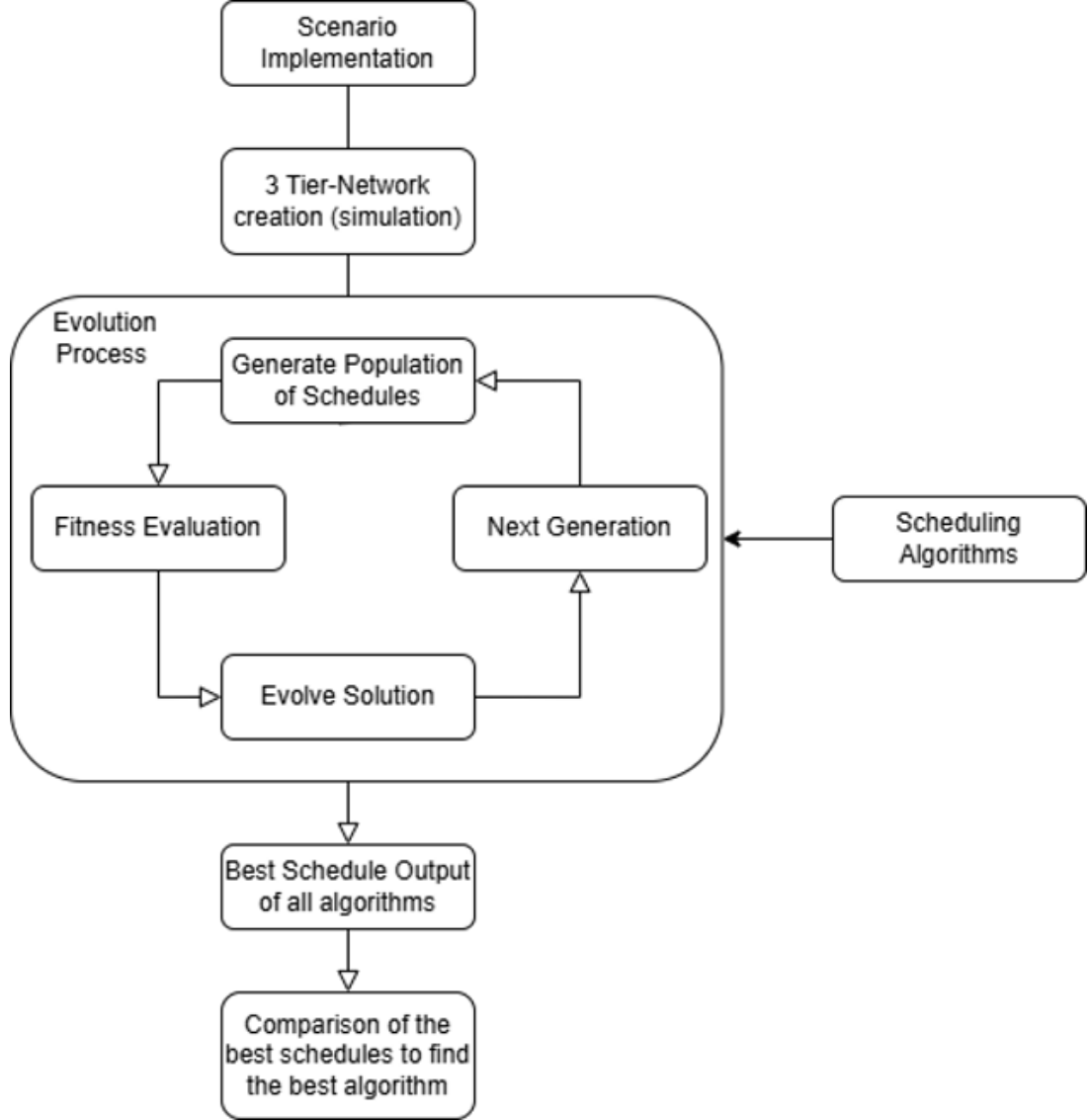


Figure 3.1: Phase 1 Architecture Diagram

3.1.3 Phase 2: Vehicular Fog Orchestration

The Phase 2 architecture shifts the focus to mobility by integrating traffic simulation with the scheduling logic.

- **SUMO + TraCI Integration:** The SUMO (Simulation of Urban MObility) engine generates vehicular movement, which is accessed via the TraCI interface to retrieve

real-time location and mobility data.

- **EdgeSimPy Simulator:** This component manages network resources and task generation, working alongside SUMO mobility information to simulate a vehicular fog computing environment.
- **Dynamic Scheduler Loop:** A specialized runtime loop that triggers at every simulation step, fetching current vehicular positions and recalculating optimal task assignments.
- **Mobility Manager:** This module applies hybrid mobility models such as Manhattan Grid for urban streets and Random Waypoint for open areas to dynamically update the network graph as vehicles move.

The architecture of the mobility-aware scheduling framework is illustrated in Figure 3.2. The diagram shows how vehicular mobility generated within the SUMO simulator is accessed through the TraCI interface and integrated into the EdgeSimPy simulation environment. This interaction enables vehicular movement to dynamically influence task generation, node proximity, and network topology within the fog computing infrastructure.

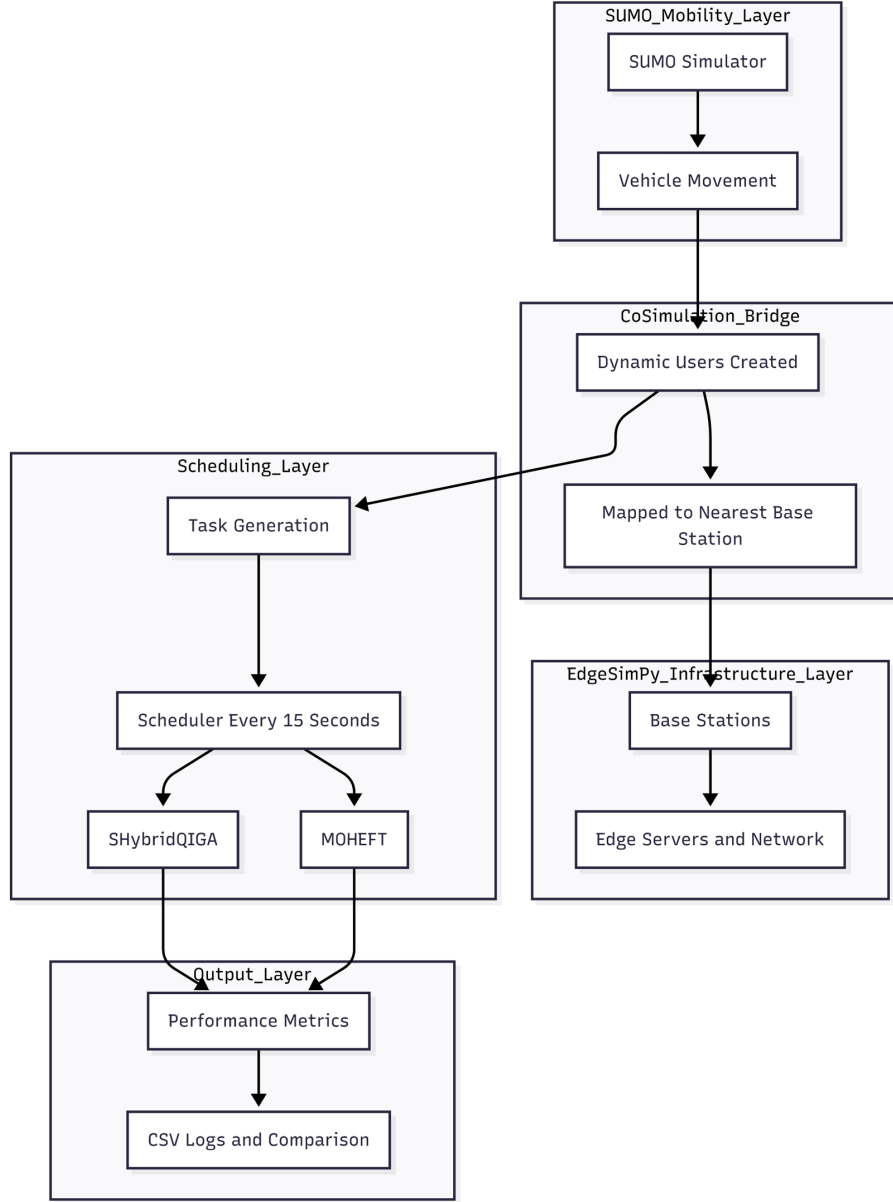


Figure 3.2: Phase 2 Architecture Diagram

3.2 Component Design

This section describes the behavioral interactions between system components through Unified Modeling Language (UML) sequence diagrams. These diagrams illustrate how computational tasks are generated, scheduled, and executed across the distributed fog computing infrastructure.

3.2.1 Behavioral Design: Sequence Diagrams

Phase 1 Sequence: Heuristic Task Offloading

The sequence diagram in Figure 3.3 illustrates the workflow of the Fog Scheduler in a stationary environment. It maps the communication between the User Interface, the Orchestration Engine, and the selected server tier, showing how fitness functions are evaluated before a task is assigned.

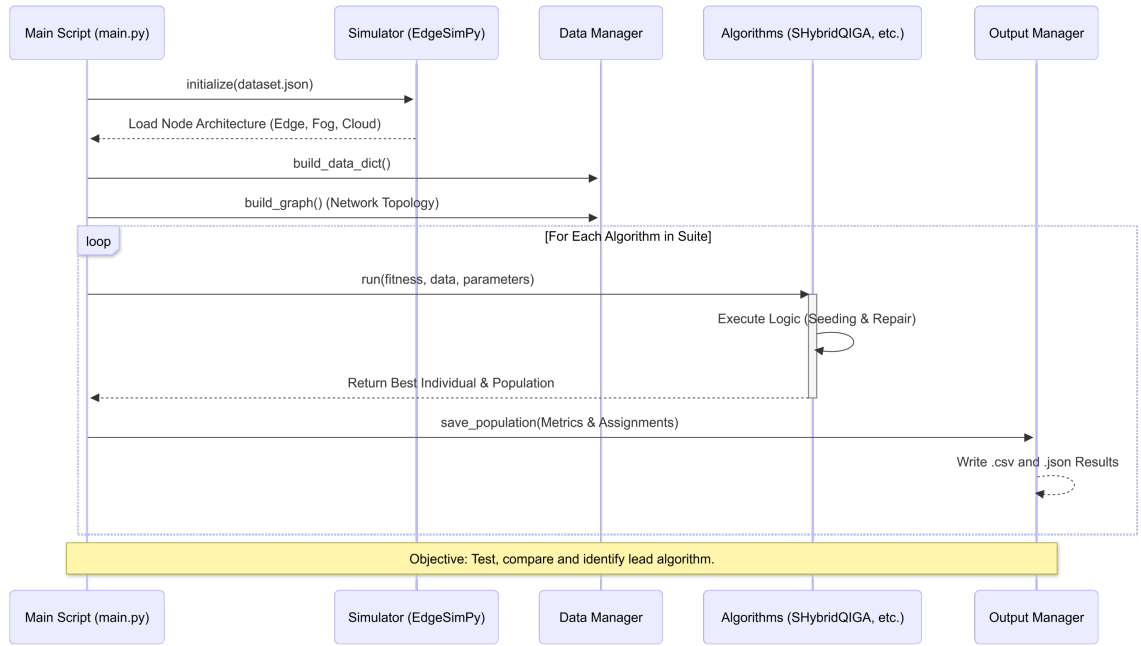


Figure 3.3: Phase 1 Sequence Diagram

Phase 2 Sequence: Dynamic Vehicular Handover

Figure 3.4 details the interactions occurring in the dynamic environment. The sequence demonstrates how the SUMO engine provides coordinate updates through the TraCI interface, how the scheduler re-evaluates optimal nodes based on vehicle proximity, and how tasks are successfully offloaded despite physical mobility and changing network conditions.

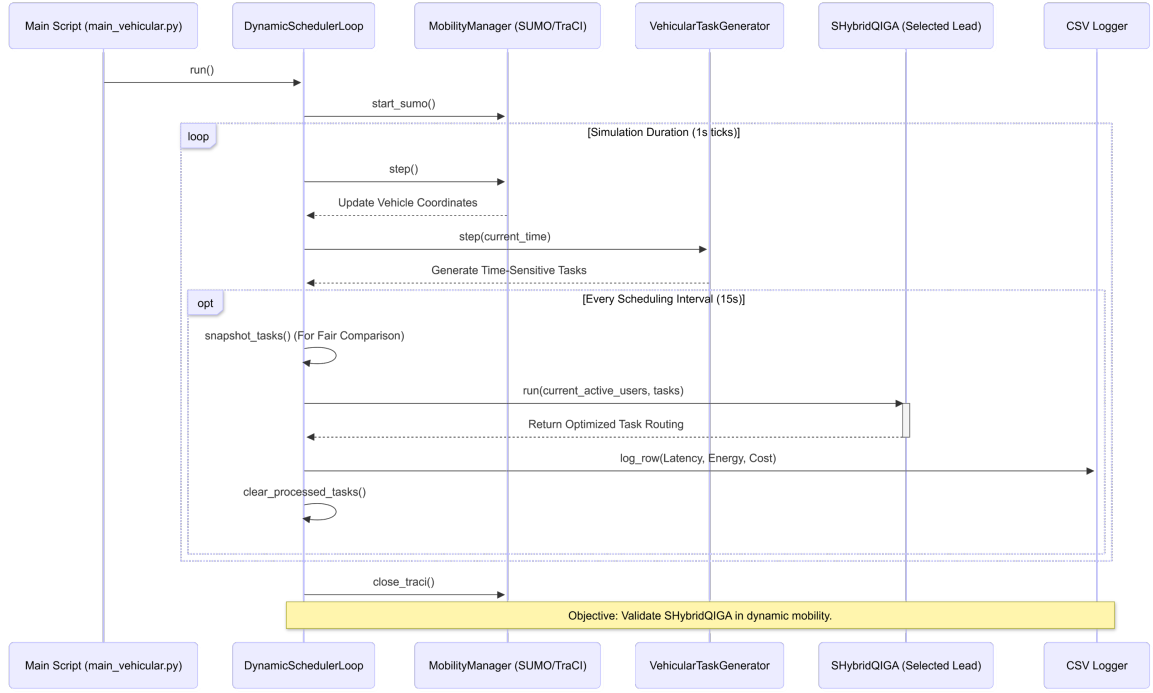


Figure 3.4: Phase 2 Sequence Diagram

3.3 Module Division

- Module 1: 3-Tier Edge-Fog-Cloud Infrastructure Modeling:** This module focuses on engineering the backend hierarchical network using the EdgeSimPy framework. It involves defining a set of m edge servers and n fog nodes, each characterized by specific computational capacities (CPU cycles/s), storage, and energy efficiency factors (ϵ). A critical sub-component is the implementation of a physics-based power model that calculates energy consumption as the product of Power (Watts) and Time (Seconds), incorporating linear, square, or cubic server power models.
- Module 2: SHybridQIGA and Multi-Objective Optimization Engine:** This module develops the Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA), which enhances standard QIGA through intelligent seeding and repair mechanisms. It utilizes qubit representation ($|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$) and adaptive rotation gates ($R(\theta)$) to optimize task routing across conflicting objectives: makespan minimization, energy reduction, and resource utilization. The engine includes a quantum observation function to map quantum states to discrete server assignments.

while maintaining solution diversity.

- **Module 3: SUMO-TraCI Vehicular Mobility Integration:** This module integrates the Simulation of Urban MObility (SUMO) engine to provide dynamic environmental data. Using the TraCI (Traffic Control Interface) protocol, the system retrieves real-time vehicle coordinates, speeds, and routes to update the network topology graph dynamically. It implements a hybrid mobility manager that switches between the Manhattan Grid Model (MGM) for urban street constraints and the Random Waypoint Model (RWM) for open areas.
- **Module 4: Digital Twin and Performance Visualization Dashboard:** This module processes simulation logs to drive a high-fidelity Digital Twin dashboard built with Streamlit and PyVis. It features "Traffic Light Logic" to visualize real-time congestion levels on edge nodes and interactive physics-enabled topology maps. The dashboard tracks granular metrics including server-side processing delay, network pathfinding latency using BFS, and cumulative energy usage across all tiers.
- **Module 5: Benchmarking and Robustness Analysis:** This module orchestrates comparative simulation tests to validate algorithmic superiority. It establishes a rigorous testing suite that benchmarks SHybridQIGA against eight alternative models, including MOHEFT (NSGA-II), Particle Swarm Optimization (PSO), and baselines like Only Cloud (OC) and Round Robin (RR). Additionally, it incorporates a robustness module that injects $\pm 10\%$ prediction noise into task weights to simulate real-world uncertainty and evaluate the stability of scheduling decisions.

3.4 Tools and Technologies

Software Requirements

- **Operating System:** Windows 10/11, Ubuntu 20.04+, or macOS
- **Programming Language:** Python 3.8 or higher
- **Simulation Libraries:** EdgeSimPy, SUMO (Simulation of Urban MObility)
- **Traffic Interface:** TraCI (Traffic Control Interface)

- **Dashboard Framework:** Streamlit
- **Visualization Tools:** PyVis, Matplotlib, NetworkX
- **Development Environment:** Visual Studio Code / PyCharm / Overleaf

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen) or equivalent AMD Ryzen
- **RAM:** 8 GB (16 GB recommended for heavy simulations)
- **Storage:** 512 GB SSD
- **Graphics:** Integrated Graphics (Dedicated GPU recommended for GUI mode)
- **Network:** High-speed Internet Connection (for library updates)

3.5 Work Breakdown and Responsibilities

The project tasks were distributed among the team members based on their areas of responsibility. Table 3.1 summarizes the contributions made by each team member.

Table 3.1: Work Breakdown and Responsibilities

Team Member	Contribution
Adithyan Ajayan	Integrated the SUMO traffic simulator into the system to introduce realistic vehicular mobility. Implemented the TraCI-based interface to retrieve vehicle coordinates and enable dynamic task generation based on moving vehicles.
Ajay Mukund A.	Designed the overall simulation framework and developed the Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA). Implemented the intelligent seeding strategy and surgical constraint-repair mechanism to optimize task scheduling across the Edge-Fog-Cloud infrastructure.
Alex Babu	Developed the performance visualization dashboard and analytics modules. Implemented scripts for processing simulation outputs and visualizing performance metrics such as energy consumption, latency, and server load.

Team Member	Contribution
Anna George	Engineered a 3-tier Edge-Fog-Cloud infrastructure using EdgeSimPy, implementing server power consumption models. Developed a physics engine to calculate energy, costs and latency. Designed multi-objective fitness functions to simultaneously optimize response times, power usage, and costs within the scheduling logic.
Bennette A. Sabu	Conducted simulation experiments and benchmarking analysis. Evaluated the proposed algorithm against baseline scheduling algorithms such as Round Robin, Genetic Algorithms, and other heuristic methods.

3.6 Key Deliverables

The successful execution of this research resulted in a suite of high-performance technical assets and experimental environments designed for next-generation Vehicular Edge Computing research.

- **3-Tier Network Infrastructure Model:** A high-fidelity simulation environment configured with hierarchical layers of Edge (Tier-1), Fog (Tier-2), and Cloud (Tier-3) resources.
- **Multi-Objective Optimization Engine:** A core scheduling module implementing fitness functions that simultaneously balance latency, energy consumption ($Power \times Time$), and monetary costs.
- **Algorithmic Benchmarking Library:** A collection of nine distinct implementations, including QIGA, HybridQIGA, MOHEFT (NSGA-II), GA, and PSO, designed for rigorous performance comparison.
- **Hardware-Constrained Scenario Generator:** A dataset creation tool that models heterogeneous hardware profiles, such as Raspberry Pi clusters and high-performance cloud servers, to test resource scarcity.
- **SUMO-TraCI Integration Module:** A cyber-physical interface that connects the SUMO traffic engine to the simulation runtime via the TraCI protocol for real-time environmental synchronization.

- **Dynamic Spatial Mapping & Handover Manager:** A coordinate translation system that maps SUMO (X/Y) vehicular positions to the network grid, enabling the tracking of mobile nodes and the execution of network handovers.
- **Heterogeneous Task Generation Engine:** A workload emulator that simulates diverse vehicular data streams, ranging from low-weight telemetry pings to computation-heavy vision analytics, during traffic bursts.
- **Continuous Runtime Scheduler Loop:** A temporal execution loop designed to run algorithms in a time-series format, validating network resilience and Service Level Agreement (SLA) adherence during unpredictable traffic spikes.

3.7 Project Timeline

The project was carried out from July 2025 to March 2026 through multiple development phases including literature survey, static system development, vehicular mobility integration, system testing, and final documentation.



Figure 3.5: Gantt Chart

3.8 Summary of Chapter

This chapter presented the architectural framework and development methodology used to construct the vehicular fog orchestration system. The system architecture integrates static fog infrastructure simulation with mobility-driven vehicular workloads generated through SUMO. By structuring the framework into modular components including infrastructure management, vehicular mobility integration, quantum-inspired scheduling, and analytics visualization, the system enables comprehensive evaluation of scheduling strategies under both controlled and dynamic network conditions. The defined deliverables and development timeline further demonstrate the systematic engineering approach used to translate theoretical quantum-inspired scheduling concepts into a mobility-aware fog computing simulation framework.

Chapter 4

Proposed Methodology: SHybridQIGA

This chapter presents the comprehensive design and mathematical formulation of the proposed Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA). The primary objective of this methodology is to orchestrate dynamic vehicular workloads across a heterogeneous edge computing infrastructure. To overcome the limitations of traditional evolutionary algorithms—which often succumb to queuing delays and energy bloat during high-traffic scenarios—the proposed model introduces a multi-core load-aware repair mechanism alongside quantum-probabilistic exploration. The following sections detail the system model, chromosome representation, mathematical fitness formulations, and the step-by-step execution pipeline of the algorithm.

4.1 System Architecture and Problem Formulation

The target environment is a highly dynamic Vehicular Edge Computing (VEC) digital twin, composed of moving vehicular nodes, Tier-1 Edge nodes (e.g., Roadside Units, Raspberry Pis), multi-core Tier-2 Fog servers, and a centralized Cloud.

4.1.1 Task and Resource Modeling

Let $T = \{t_1, t_2, \dots, t_n\}$ represent a dynamic batch of n vehicular tasks generated within a specific scheduling interval. Each task t_i is characterized by its computational weight (instructions), data size (bytes), and a strict Service Level Agreement (SLA) deadline.

Let $R = \{r_1, r_2, \dots, r_m\}$ represent the available computational resources (servers). Each resource r_j possesses a specific number of CPU cores (C_j), a base clock frequency (F_j), memory capacity, and a defined power model (static and dynamic power coefficients).

4.1.2 Delay Calculation (Multi-Core Physics Engine)

The total processing delay for a task t_i assigned to server r_j is the sum of its network path delay and CPU execution delay. The network path delay (D_{path}) is calculated dynamically using Dijkstra's shortest path algorithm, computing the transmission time from the vehicle's associated base station to the server's base station based on the task's data size and network bandwidth.

The execution delay (D_{exe}) relies on a multi-core load-aware formula. Traditional algorithms assume constant server speeds, leading to severe bottlenecks. In the proposed physical model, if the number of currently executing tasks (L_j) on server r_j is less than or equal to its total CPU cores (C_j), the tasks run in parallel without performance degradation. However, if the server is overloaded ($L_j > C_j$), the effective CPU frequency is proportionally divided among the tasks:

$$F_{effective,j} = \begin{cases} F_j & \text{if } L_j \leq C_j \\ \frac{C_j \times F_j}{L_j} & \text{if } L_j > C_j \end{cases} \quad (4.1)$$

The execution delay is then defined as:

$$D_{exe} = \frac{\text{Weight}(t_i)}{F_{effective,j}} \quad (4.2)$$

4.1.3 Multi-Objective Fitness Function

To ensure the orchestrator satisfies both QoS and infrastructure sustainability constraints, a single-objective Weighted-Sum fitness function is utilized to evaluate chromosome viability. This $O(N)$ evaluation approach ensures millisecond-level scheduling responsiveness compared to computationally heavy $O(N^2)$ Pareto-sorting techniques like NSGA-II.

For a given mapping solution, the normalized system Energy (E_{norm}), Latency (L_{norm}), and Financial Cost (C_{norm}) are aggregated using predefined weights ($W_{lat} = 0.7$, $W_{energy} = 0.2$, $W_{cost} = 0.1$). Furthermore, a strict penalty multiplier (P_{SLA}) is added for every task that exceeds its strict deadline or breaches server memory limits:

$$\text{Fitness} = (W_{energy} \cdot E_{norm}) + (W_{lat} \cdot L_{norm}) + (W_{cost} \cdot C_{norm}) + P_{SLA} \quad (4.3)$$

The algorithm's overarching objective is to minimize this fitness score.

4.2 Proposed Algorithm: SHybridQIGA

The proposed SHybridQIGA integrates three major computational phases: Quantum Superposition for rapid global exploration, Heuristic Seeding for optimized starting conditions, and a Deterministic Multi-Core Repair for localized constraint satisfaction.

4.2.1 Quantum Chromosome Representation

Unlike traditional genetic algorithms that use binary arrays (0s and 1s), SHybridQIGA utilizes Quantum bits (Q-bits) to represent the population. A Q-bit represents a superposition of states, allowing a single quantum chromosome to mathematically represent multiple potential task-to-server mappings simultaneously.

A Q-bit is defined by a pair of complex numbers (α, β) , where $|\alpha|^2 + |\beta|^2 = 1$. The values $|\alpha|^2$ and $|\beta|^2$ represent the probability of the Q-bit collapsing to state 0 or 1, respectively. The entire network mapping for n tasks across m resources is represented by a quantum chromosome array initialized with equal probabilities $\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$.

4.2.2 Heuristic Seeding Initialization

To accelerate convergence and prevent the algorithm from wasting generations exploring highly inefficient solutions, SHybridQIGA injects a heuristically optimized "Seed" chromosome into the population at Generation 0.

The seeding mechanism sorts all available Tier-1 and Tier-2 resources by their operational monetary cost. To prevent "poisoning" the seed by overloading a single cheap server, the algorithm identifies the top 50% most cost-efficient servers ($K = \max(3, m/2)$). It then distributes the initial batch of tasks in a round-robin fashion strictly among these top K servers. This instantly anchors the quantum population toward a low-cost, energy-efficient baseline before evolutionary mutations occur.

4.2.3 Quantum Measurement and Evolutionary Rotation

During evaluation, the continuous quantum chromosomes are "measured" (collapsed) into classical binary arrays to calculate their physical fitness.

To evolve the population toward the optimal solution, SHybridQIGA utilizes Quantum Rotation Gates. The probability amplitudes (α, β) of each Q-bit are updated based on the

fitness of the best-performing chromosome in the current generation. The rotation gate shifts the phase of the Q-bit by an angle $\Delta\theta$, increasing the probability that subsequent measurements will yield the optimal trait. The rotation matrix is defined as:

$$\begin{bmatrix} \alpha' \\ \beta' \end{bmatrix} = \begin{bmatrix} \cos(\Delta\theta) & -\sin(\Delta\theta) \\ \sin(\Delta\theta) & \cos(\Delta\theta) \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \quad (4.4)$$

To prevent premature convergence and local-minima entrapment, the rotation angle θ utilizes an adaptive dynamic decay factor. It begins at a highly exploratory state (0.05π) and exponentially decays to a fine-tuning exploitative state (0.01π) as the generations progress. Additionally, the algorithm continuously monitors genetic diversity by evaluating the number of unique fitness costs in the population. If the population stagnates (e.g., fewer than 5 unique cost profiles), the base mutation rate is dynamically spiked to force the algorithm to explore new topological assignments.

4.3 The Multi-Core Load-Aware Repair Mechanism

The defining characteristic that distinguishes SHybridQIGA from standard evolutionary baselines is its intelligent, deterministic repair mechanism. Traditional meta-heuristics often generate invalid chromosomes where tasks are dumped onto cheap but overloaded nodes, resulting in massive SLA violations.

To rectify this, the repair mechanism acts as a highly targeted post-processing filter applied to every classical chromosome before fitness evaluation. It evaluates constraints and surgically mutates specific genes that violate system thresholds. The protocol executes the following logical flow:

1. **Dynamic Load Tracking:** The mechanism decodes the current classical 1D chromosome array (**CInd**), accurately tracking the active queue depth (L_j) on every server r_j in the network using an internal state dictionary.
2. **Bottleneck Detection:** For every task assigned to a server, the mechanism calculates the estimated execution delay using the multi-core scaling formula (Eq. 2.1). If the task's total delay (Execution + Path) exceeds its designated SLA deadline, the specific gene block assigning that task is flagged as a bottleneck violation.

3. Strategic Spill-over (Constraint Satisfaction):

- If the bottleneck is caused by a massive *Path Delay*, the algorithm scans for the closest available Tier-1 base station with an open CPU core.
- If the bottleneck is caused by a massive *Execution Delay* (e.g., a Tier-1 Raspberry Pi queue is overloaded), the algorithm scans for a higher-tier, multi-core Fog server. It verifies that the target server has sufficient idle cores ($L_{target} \leq C_{target}$) to absorb the task without triggering a secondary bottleneck.

4. **Gene-Level Chromosome Mutation:** Once a valid target server is identified, the repair mechanism surgically mutates the classical chromosome array. It calculates the exact index offset of the task ($i \times m$ resources), flips the binary gene of the overloaded server from $1 \rightarrow 0$, and sets the binary gene of the optimal target server to 1. The internal load trackers are immediately updated ($L_{source} - 1, L_{target} + 1$) to ensure subsequent task evaluations within the same chromosome accurately recognize the newly migrated load.

5. **Green Edge Clamp:** To prevent the algorithm from unnecessarily waking up high-power servers for lightweight tasks, a final cost-minimization check is executed. If a lightweight task is assigned to an expensive Tier-2 server but has a highly lenient deadline, its gene is mutated to downgrade the task back to a cheaper Tier-1 node, provided that the cheaper node's queue is under a safe capacity threshold ($< 200\%$).

4.4 Summary of the Algorithmic Workflow

The overall execution pipeline of SHybridQIGA during a dynamic scheduling epoch operates as follows:

1. A dynamic batch of n unassigned vehicular tasks is retrieved from the environment.
2. The Quantum Population is initialized, and the Heuristic Seed chromosome is generated based on cost-efficiency profiles.
3. The Quantum chromosomes are measured (collapsed) to produce Classical binary solutions.

4. The Multi-Core Load-Aware Repair Mechanism inspects each classical solution, eliminating SLA-violating bottlenecks via targeted gene mutations and enforcing power efficiency via the Green Edge Clamp.
5. The Weighted-Sum multi-objective fitness function evaluates the newly repaired population.
6. The global best solution is updated, and the Quantum Rotation Gates (θ) adjust the probability amplitudes of the quantum population toward the best trajectory.
7. Steps 3-6 repeat until the maximum generation limit is reached, after which the optimal routing matrix is dispatched to the physical network infrastructure for execution.

By bridging rapid quantum-probabilistic exploration with a physically-grounded, multi-core repair protocol, SHybridQIGA ensures rapid decision making while sustaining Pareto-optimal adherence to stringent vehicular QoS constraints and telecommunication energy limits.

Chapter 5

Results and Discussions

This chapter presents the performance evaluation of the proposed SHybridQIGA against quantum evolutionary algorithms, standard evolutionary algorithms, and normal baseline algorithms to prove its efficacy using a heuristic algorithmic approach. The primary goal of this evaluation is to reduce task execution and offloading latency while closely meeting real deadlines, all while considering energy consumption and operating costs to make fog computing operations more sustainable. We employed a weighted fitness function that prioritizes latency ($w_{\text{latency}} = 0.7$), with cost weighted at 0.1 and energy consumption at 0.2. The following sections detail the analysis of the performance based on these three metrics across various distinct scenarios.

5.1 Phase 1: Static Heuristic Evaluation

In this first phase of testing, the performance was evaluated across five distinct scenarios. The evaluation was based on three main metrics: average energy consumption (Joules), average latency, and cost. Based on these metrics, by assigning weights prioritizing latency, we compared the weighted fitness score and then identified the best algorithm.

5.1.1 Latency vs. Energy Trade-offs

We observed a clear trade-off between completion speed and energy efficiency in both the critical real-time (RT) and compute-heavy scenarios, which differ primarily in the tighter deadline of the RT workload. As expected, the cloud-centric OC baseline achieved the lowest raw latency (≈ 6.0 s) due to its high computational capacity, followed by GA and MOHEFT, which also prioritize fast completion. The greedy MOHEFT scheduler achieved low latency (≈ 6.8 – 7.0 s) by selecting the earliest finishing time without considering resource cost or energy usage. In contrast, the proposed SHybridQIGA achieved

slightly higher latency ($\approx 8.6\text{--}9.2$ s) while significantly reducing energy consumption and operational cost, highlighting its effectiveness as a balanced multi-objective scheduling strategy.

Table 5.1 presents the performance comparison in the critical real-time scenario. While MOHEFT completed task offloading faster (6.82 s), this performance came at a substantial energy cost of 437.16 J compared to only 31.34 J required by SHybridQIGA. Furthermore, while the baseline quantum-inspired algorithms (HybridQIGA and QIGA) achieved highly competitive latency values (8.42 s and 8.04 s respectively), their energy consumption was more than double that of SHybridQIGA ($\approx 70\text{--}73$ J). As a result, SHybridQIGA achieved the best overall weighted fitness score (12.92) compared to its closest competitor HybridQIGA (20.66) and the greedy MOHEFT approach (93.68).

Table 5.1: Performance Comparison in Critical Real-Time Scenario

Algorithm	Avg. Cost	Avg. Energy (J)	Avg. Latency (s)	Weighted Score
SHybridQIGA	6.27	31.34	8.61	12.92
HybridQIGA	7.04	70.29	8.42	20.66
QIGA	6.81	73.23	8.04	20.95
OE	8.14	138.41	7.90	34.03
RR	8.80	156.83	7.59	37.56
RA	9.18	164.12	7.74	39.16
SQIGA	8.93	187.55	7.29	43.50
PSO	9.18	192.72	7.30	44.57
DE	9.30	211.95	7.15	48.33
GA	12.39	347.03	6.39	75.11
MOHEFT	14.80	437.16	6.82	93.68
OC	32.88	690.38	6.00	145.56

This trend continues in the compute-heavy scenario (Table 5.2 and Fig. 5.1), where SHybridQIGA achieved a weighted score of 14.16. In this scenario, HybridQIGA emerged as the closest competitor with a score of 18.55; however, SHybridQIGA’s superior energy

optimization (34.96 J vs 60.07 J) safely maintained its lead. Traditional evolutionary algorithms like GA (73.05) and greedy heuristics like MOHEFT (106.32) fell far behind due to their inability to effectively manage energy overheads in compute-intensive environments.

Table 5.2: Performance Comparison in Compute-Heavy Scenario

Algorithm	Avg. Cost	Avg. Energy (J)	Avg. Latency (s)	Weighted Score
SHybridQIGA	6.92	34.96	9.25	14.16
HybridQIGA	6.76	60.07	8.38	18.55
QIGA	7.19	81.18	8.19	22.69
OE	8.11	125.40	8.21	31.63
RR	8.56	134.93	7.87	33.35
RA	9.20	150.13	8.10	36.62
PSO	8.72	166.26	7.59	39.44
DE	8.93	178.91	7.54	41.96
SQIGA	9.06	186.02	7.51	43.37
GA	12.63	335.55	6.68	73.05
MOHEFT	18.14	497.99	7.01	106.32
OC	33.69	707.44	6.01	149.06

In the balanced scenario (Table 5.3 and Fig. 5.2), designed to represent normal daily workloads, SHybridQIGA further demonstrated its ability to obtain a well-optimized schedule across all three metrics, achieving a weighted score of 9.83. It is worth noting that the standard quantum-inspired variants, HybridQIGA (11.58) and QIGA (12.06), also performed exceptionally well here compared to traditional evolutionary techniques like GA (21.87). This validates the overarching effectiveness of the quantum-probabilistic approach. However, the smart repair mechanism of SHybridQIGA provided the final edge, successfully prioritizing low-latency Tier 1 and Tier 2 nodes to satisfy strict latency requirements while minimizing cost.

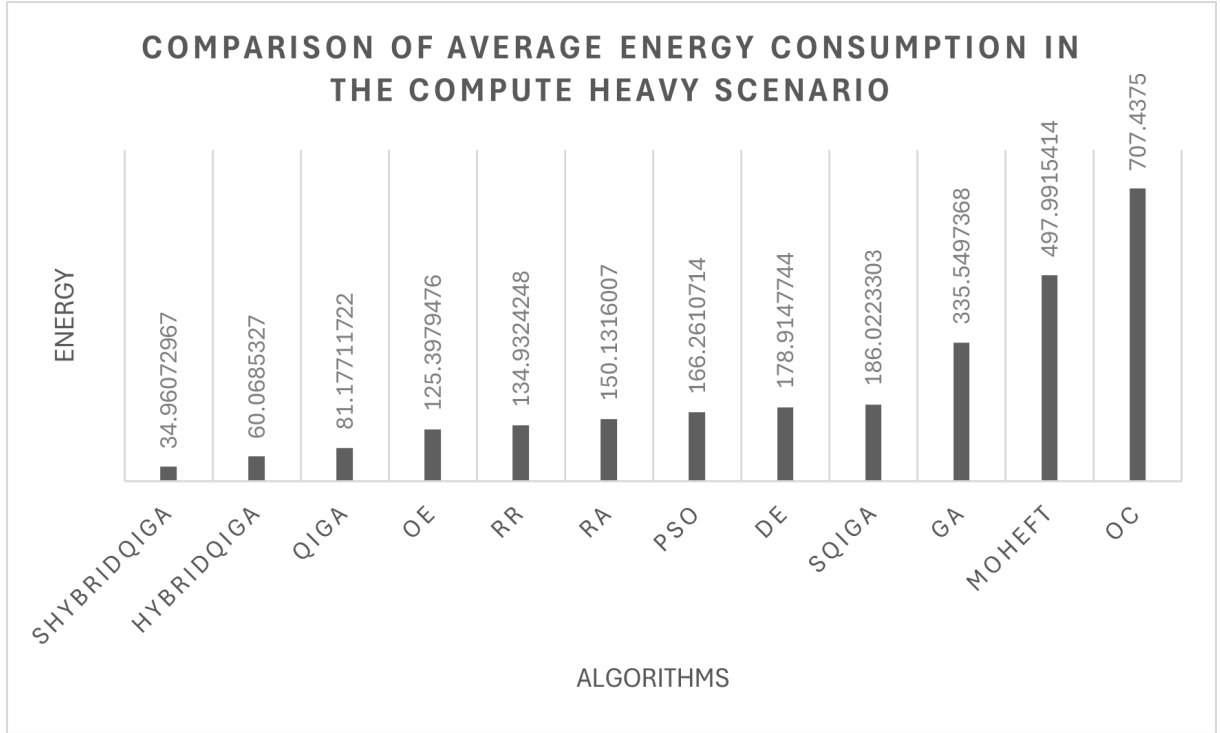


Figure 5.1: Comparison of average energy consumption in the compute-heavy scenario

Table 5.3: Performance Comparison in Balanced Scenario

Algorithm	Avg. Cost	Avg. Energy (J)	Avg. Latency (s)	Weighted Score
SHybridQIGA	2.79	13.72	9.72	9.83
HybridQIGA	2.75	23.67	9.39	11.58
QIGA	2.81	26.47	9.26	12.06
OE	3.16	41.36	9.38	15.16
SQIGA	3.14	46.24	9.00	15.86
RR	3.26	46.06	9.32	16.06
DE	3.14	47.56	8.98	16.11
RA	3.66	53.04	9.35	17.52
PSO	3.38	55.13	8.98	17.65
GA	3.68	77.54	8.56	21.87
MOHEFT	8.19	244.86	9.01	56.10
OC	13.62	286.12	8.95	64.85

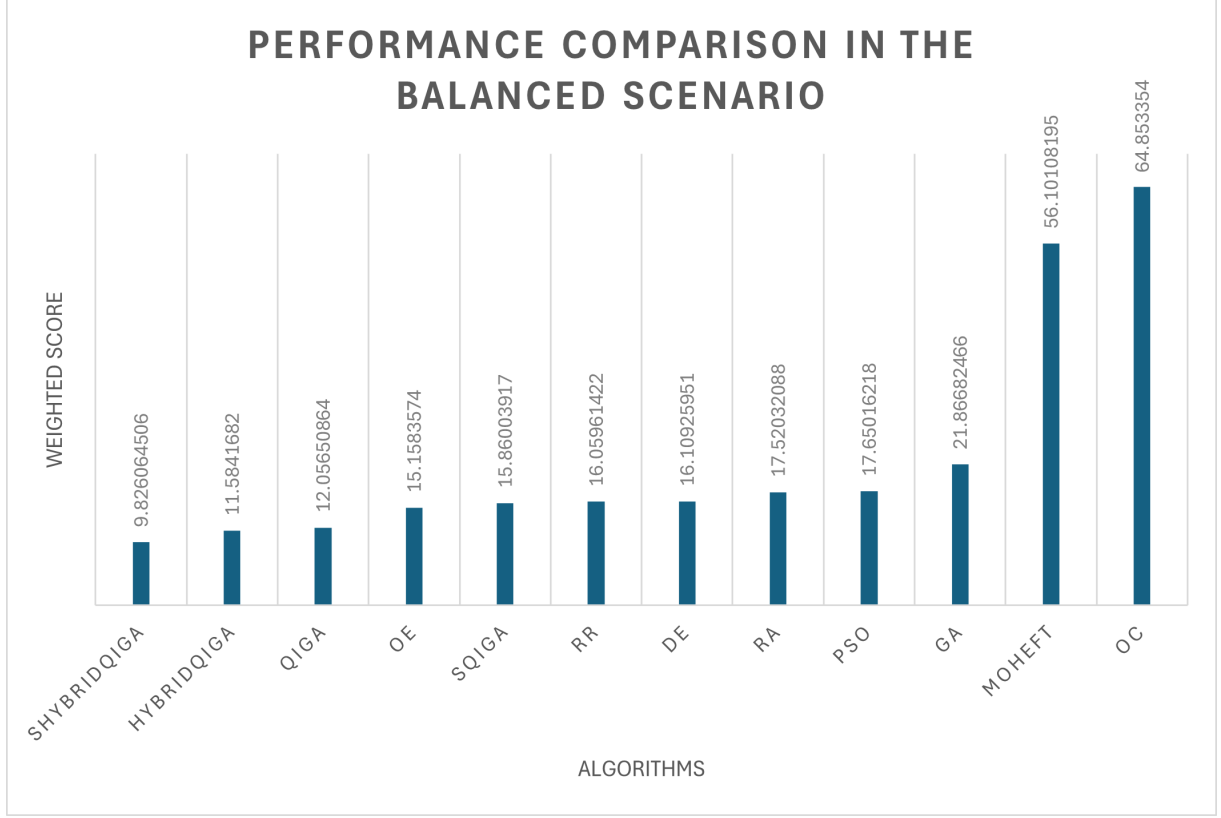


Figure 5.2: Performance comparison in the balanced scenario

5.1.2 Limitations under Network Saturation

Regarding drawbacks and limitations, SHybridQIGA faced challenges in heavy data and high-traffic scenarios.

When bandwidth was constrained in the heavy data scenario (Table 5.4), MOHEFT outperformed SHybridQIGA (score 32.72 vs. 37.20). This demonstrated that the greedy nature of MOHEFT was more effective in minimizing data transmission time compared to the heuristic approach. In this constrained environment, SHybridQIGA was closely followed by its non-smart counterparts HybridQIGA (38.07) and QIGA (38.62), while standard GA lagged slightly behind (39.69).

When the load was further increased in the high-traffic scenario (Table 5.5 and Fig. 5.3), SHybridQIGA's performance degraded slightly. Under these conditions, the baseline HybridQIGA (14.48) and QIGA (15.65) adapted better to the extreme request saturation. However, SHybridQIGA still maintained a competitive score of 16.08, narrowly edging out simple round-robin (RR) distribution (16.09) and remaining vastly superior to GA (24.86) and MOHEFT (153.26), which struggled immensely with traffic bottlenecks.

Table 5.4: Performance Comparison in Heavy Data Scenario

Algorithm	Avg. Cost	Avg. Energy (J)	Avg. Latency (s)	Weighted Score
MOHEFT	2.38	10.70	43.34	32.72
SHybridQIGA	2.36	11.38	49.55	37.20
HybridQIGA	2.20	16.93	49.23	38.07
QIGA	2.31	19.96	49.13	38.62
DE	2.44	26.55	47.92	39.10
SQIGA	2.44	28.20	48.19	39.62
GA	2.52	37.27	45.70	39.69
OE	2.55	32.82	49.40	41.40
RR	2.65	33.99	49.49	41.70
PSO	2.90	41.00	48.52	42.45
RA	2.98	43.62	49.56	43.71
OC	11.06	232.31	52.55	84.36

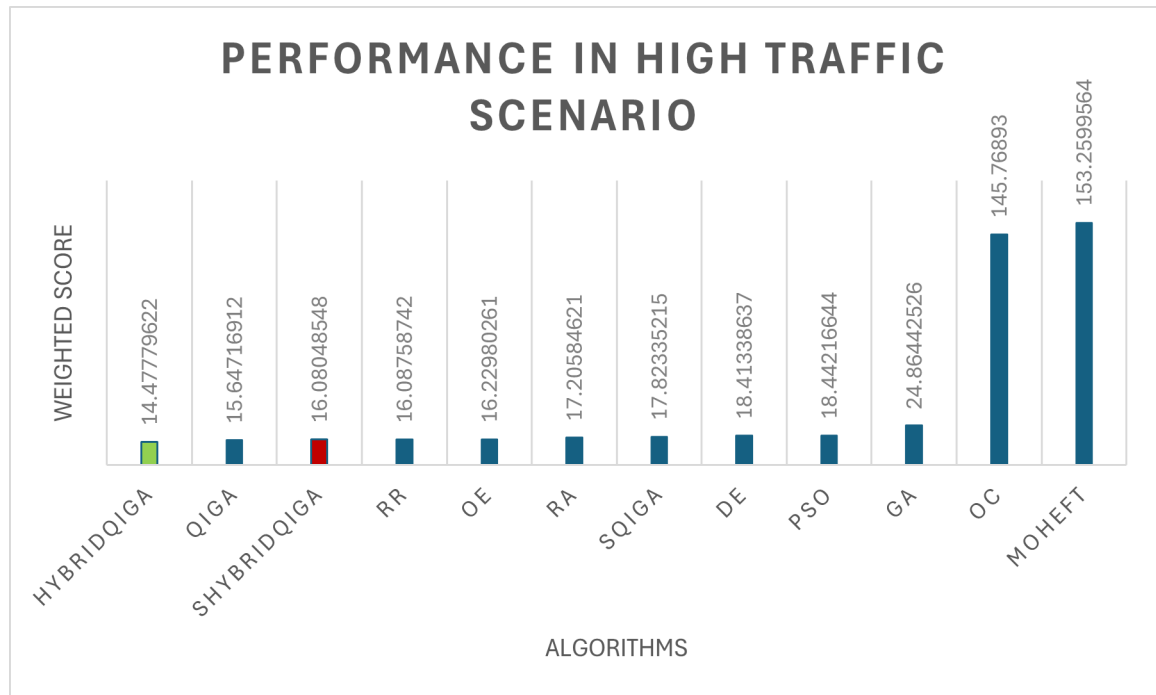


Figure 5.3: Weighted fitness score comparison in the high-traffic scenario

Table 5.5: Performance Comparison in High Traffic Scenario

Algorithm	Avg. Cost	Avg. Energy (J)	Avg. Latency (s)	Weighted Score
HybridQIGA	5.66	43.73	7.38	14.48
QIGA	5.06	53.37	6.38	15.65
SHybridQIGA	8.33	39.15	10.60	16.08
RR	5.01	56.35	6.17	16.09
OE	5.41	54.18	6.93	16.23
RA	5.46	60.19	6.60	17.21
SQIGA	5.21	65.41	6.03	17.82
DE	5.22	68.76	5.91	18.41
PSO	5.22	68.70	5.97	18.44
GA	5.61	103.79	5.06	24.86
OC	32.96	692.14	5.78	145.77
MOHEFT	23.97	719.05	10.08	153.26

5.2 Phase 2: Dynamic Vehicular Edge Evaluation

While Phase 1 evaluated the algorithms under controlled, static heuristic workloads, Phase 2 introduces a significantly more realistic evaluation framework by integrating the *edge_sim_py* library with the SUMO (Simulation of Urban MObility) traffic simulator. This digital twin environment enables the modeling of physical vehicular mobility, dynamic high-frequency task generation, and network handovers across base stations over a continuous 120-second simulation window.

This dynamic environment introduces severe operational challenges. Vehicles continuously move between network coverage zones, causing physical communication delays and resource availability to fluctuate. Furthermore, as vehicular density increases, the network experiences aggressive bursts of up to 800 simultaneous computational tasks, testing whether scheduling algorithms can dynamically avoid traffic bottlenecks or if they succumb to queuing failures under system pressure.

To analyze algorithmic scalability, four distinct infrastructure topologies were evaluated. The performance of SHybridQIGA was benchmarked against traditional Meta-Heuristics (GA, QIGA), greedy edge heuristics (MOHEFT), and a centralized Cloud baseline (OC).

5.2.1 Standardized Traffic Injection Profile

To ensure absolute fairness, all infrastructural scenarios were subjected to the exact same dynamic vehicular workload generated by SUMO.

As illustrated in Figure 5.4, the simulation models a progressive "Rush Hour". This ensures algorithms are evaluated during low-load conditions (testing energy minimization) and extreme high-load conditions (testing deadline preservation and bottleneck avoidance).

5.2.2 Scenario 1: Baseline Urban Grid

The first scenario represents a balanced baseline infrastructure (20 Tier-1, 5 Tier-2, 1 Cloud). Algorithms must intelligently decide when to utilize local, low-power Tier-1 edge nodes versus offloading to heavier, multi-core Tier-2 fog servers.

As shown in Table 5.6 and the associated time-series plots, standard evolutionary

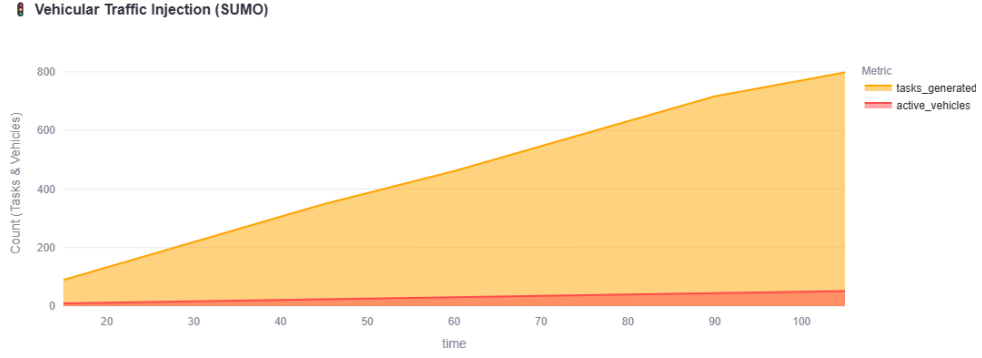


Figure 5.4: Standardized vehicular traffic injection profile used across all scenarios. Vehicular density gradually scales, creating a massive influx of over 800 dynamically generated tasks at peak congestion ($t = 105s$).

Table 5.6: Cumulative Performance in the Baseline Urban Grid

Algorithm	Missed Deadlines	Avg. Latency (s)	Total Energy (J)	Total Cost (\$)
SHybridQIGA	166	1.08	156,226.5	7,920.81
GA	394	1.53	110,724.9	6,891.24
MOHEFT	403	1.09	296,895.4	14,104.42
QIGA	489	1.67	91,100.2	6,888.45
OC (Cloud)	691	1.96	1,293,495.0	61,595.00



Figure 5.5: SLA Violation Rate over time.

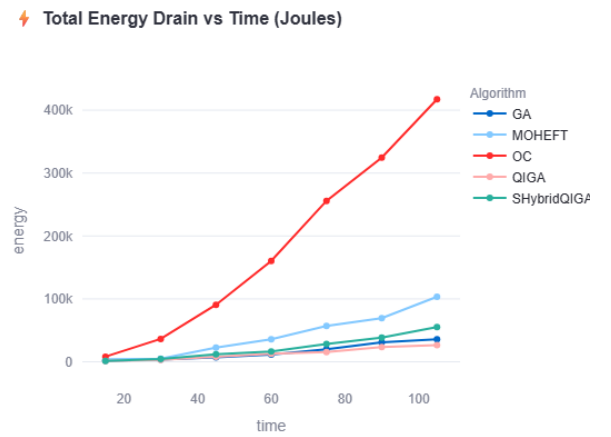


Figure 5.6: Total Energy Drain over time.

algorithms such as GA and QIGA exhibit rapidly increasing SLA violations as traffic intensifies. This occurs because traditional models blindly queue tasks onto weak, single-core Tier-1 nodes to minimize raw energy formulas. Consequently, tasks sit in the CPU queue, timeout, and physically fail. Conversely, SHybridQIGA’s multi-core load-aware mechanism actively detects these queuing bottlenecks and strategically ”spills over” traffic to the multi-core Tier-2 servers. This strategic trade-off allowed SHybridQIGA to significantly reduce SLA violations (166 vs GA’s 394) while simultaneously maintaining a highly efficient energy profile, expending 156,226.5 Joules to rescue critical tasks compared to the greedy MOHEFT algorithm’s highly wasteful 296,895.4 Joules.

5.2.3 Scenario 2: Edge-Heavy Infrastructure

In this scenario, the environment is highly saturated with cheap, low-power Tier-1 nodes (35 Tier-1, 5 Tier-2, 1 Cloud) but lacks heavy multi-core fog servers.

Table 5.7: Cumulative Performance in the Edge-High Infrastructure

Algorithm	Missed Deadlines	Avg. Latency (s)	Total Energy (J)	Total Cost (\$)
SHybridQIGA	153.5	0.96	103,317.7	5,511.53
GA	330.0	1.24	55,475.8	4,658.77
MOHEFT	368.5	0.95	190,950.1	9,590.02
QIGA	376.0	1.34	47,160.1	4,772.39
OC (Cloud)	699.0	1.98	1,361,296.8	64,823.65

This topology acts as a trap for traditional meta-heuristics. Due to the vast number of available edge nodes, algorithms like GA and QIGA struggle mathematically to identify which specific nodes are physically closest to the vehicles, resulting in higher latency. SHybridQIGA effortlessly navigated this trap. By utilizing Quantum Superposition, it globally explored the vast array of Tier-1 nodes simultaneously, identifying optimal localized assignments. This allowed it to maintain a near 1-second latency (0.96s) and drastically reduce SLA violations compared to both GA and greedy heuristics, missing only 153.5 deadlines while restricting energy drain to 103,317.7 Joules.

5.2.4 Scenario 3: Server-Heavy Infrastructure

This scenario represents a premium deployment featuring an abundance of heavy, multi-core Tier-2 Fog servers (25 Tier-1, 15 Tier-2, 1 Cloud). It specifically tests the algorithms' ability to manage power-hungry hardware.

Table 5.8: Cumulative Performance in the Server-High Infrastructure

Algorithm	Missed Deadlines	Avg. Latency (s)	Total Energy (J)	Total Cost (\$)
SHybridQIGA	167	1.00	119,751.6	5,882.62
GA	311	1.21	93,336.2	5,337.68
MOHEFT	359	0.94	198,878.2	9,174.61
QIGA	373	1.37	71,892.1	5,369.50
OC (Cloud)	685	1.99	1,367,351.4	65,111.97

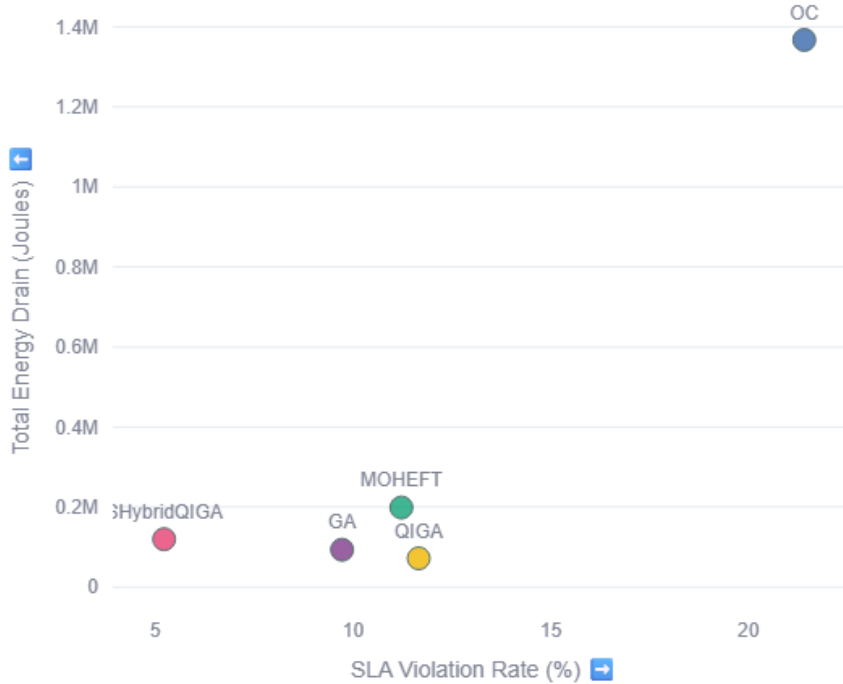


Figure 5.7: Pareto trade-off between Energy Drain and SLA Violations in the Server-High scenario. Algorithms in the bottom-left represent the optimal Pareto frontier.

As shown in Figure 5.7, this configuration exposes the fatal flaw in greedy heuristics like MOHEFT. Operating on an "Earliest Finish Time" logic, MOHEFT routed almost

all vehicular traffic to the fastest Tier-2 servers until they overloaded, completely ignoring local efficiency. This caused MOHEFT to burn nearly 200,000 Joules (198,878.2 J). SHybridQIGA acted as a perfect load-balancer. It calculated the exact number of cores available and strictly enforced a sustainability cost clamp, successfully restricting energy drain to 119,751.6 Joules while securing an ultra-low latency of 1.00 seconds and only 167 SLA violations, vastly outperforming GA (311 violations).

5.2.5 Scenario 4: Ultra-Dense Infrastructure

The final scenario represents a dense 6G deployment where computational resources are highly abundant (55 Tier-1, 15 Tier-2, 1 Cloud), testing the pure mathematical search speed of the algorithms.

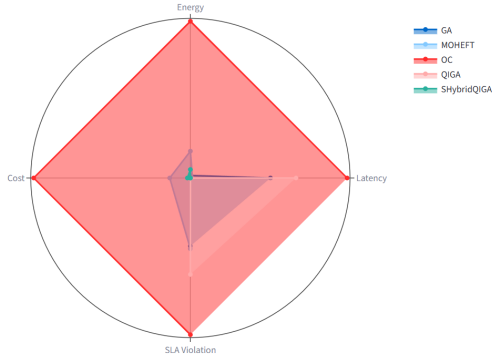
Table 5.9: Cumulative Performance in the Increased Infrastructure Scenario

Algorithm	Missed Deadlines	Avg. Latency (s)	Total Energy (J)	Total Cost (\$)
SHybridQIGA	134	0.82	49,429.3	2,920.39
GA	216	0.91	35,294.1	2,700.55
MOHEFT	252	0.77	94,728.6	4,873.58
QIGA	259	0.97	32,009.3	2,808.27
OC (Cloud)	670	1.94	1,285,263.0	61,203.00

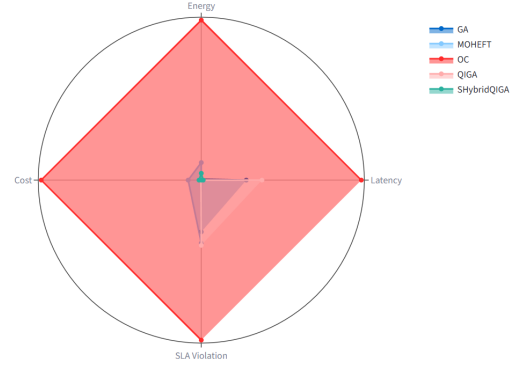
With virtually infinite resources, SHybridQIGA’s heuristic seeding approach instantly anchored the quantum population in highly efficient regions at Generation 0. This allowed the algorithm to map tasks to optimal local nodes instantly, successfully breaking the sub-second latency barrier (0.82s) and minimizing SLA violations to an absolute low of 134.

5.2.6 Cross-Scenario Algorithmic Footprint Analysis

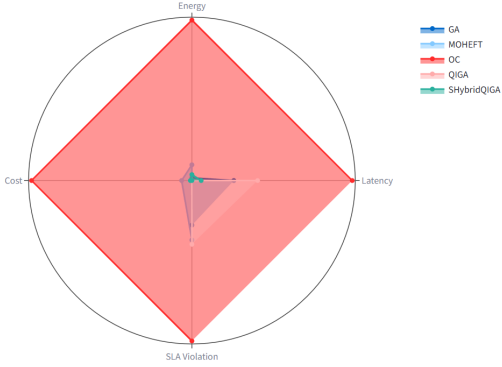
To visualize the multi-objective optimization capabilities of the algorithms across varying infrastructures, Figure 5.8 presents the algorithmic footprint (Radar Charts) for all four scenarios. The axes represent normalized values (0 to 1) for Latency, Energy, Cost, and SLA Violations. A smaller polygon surface area denotes a more highly optimized, Pareto-efficient algorithm.



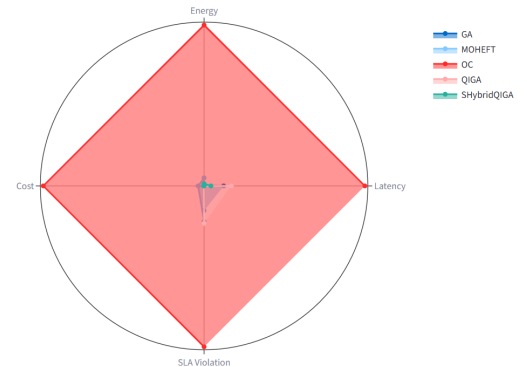
(a) Scenario 1: Base Case



(b) Scenario 2: Edge-Heavy



(c) Scenario 3: Server-Heavy



(d) Scenario 4: Ultra-Dense

Figure 5.8: Multi-objective Radar Chart comparison across all four infrastructure deployments. SHybridQIGA consistently maintains the smallest surface area, proving its robustness against network topology changes.

As evident in the sub-figures, algorithms like MOHEFT and the Cloud (OC) exhibit massive spikes in Energy and Latency, respectively. Standard GA performs adequately in standard conditions but expands significantly during the Server-High scenario (c). Conversely, SHybridQIGA tightly hugs the center axis across every single infrastructure topology, mathematically proving that its multi-core load-aware quantum repair mechanism is highly adaptable and infrastructure-agnostic.

5.3 Chapter Summary

This chapter presented a comprehensive evaluation of the proposed SHybridQIGA scheduling framework across two experimental phases. In Phase 1, heuristic-based experiments were conducted under multiple static workload scenarios to analyze the trade-offs between latency, energy consumption, and operational cost. The results showed that SHybridQIGA achieved the best weighted performance in the majority of scenarios, reducing average energy consumption by 93% compared to MOHEFT and by 50% against baseline quantum models.

Phase 2 extended the evaluation to a highly dynamic Vehicular Edge Computing digital twin. Under varying infrastructure topologies and extreme vehicular traffic jams, SHybridQIGA proved its dominance as a Pareto-optimal orchestrator. By utilizing a multi-core load-aware repair mechanism to dynamically bypass network bottlenecks, it slashed Service Level Agreement (SLA) violations by up to 58% compared to standard GA and up to 59% against MOHEFT. By strategically waking up powerful fog nodes only to clear traffic, SHybridQIGA paid a calculated energy premium to rescue failing deadlines, yet still consumed up to 48% less energy than greedy heuristics, establishing it as a highly robust and scalable scheduling strategy for next-generation sustainable edge networks.

Chapter 6

Conclusions & Future Scope

6.1 Conclusion

This project successfully addressed the critical challenges of dynamic resource allocation, strict latency adherence, and energy management in Vehicular Edge Computing (VEC) environments. By introducing the Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA), the study bridged the rapid global exploration capabilities of quantum computing principles with a deterministic, multi-core load-aware repair mechanism. This novel approach effectively mitigates the severe queuing delays and energy bloat typically associated with traditional evolutionary algorithms, preventing system failures during high-traffic vehicular scenarios.

Comprehensive evaluations across both static heuristic workloads and dynamic, digital twin simulations (utilizing SUMO and *edge_sim_py*) conclusively validated the framework's superiority. SHybridQIGA consistently achieved Pareto-optimal performance across varying infrastructure densities, drastically reducing Service Level Agreement (SLA) violations by up to 59% by intelligently routing tasks around network bottlenecks. Furthermore, by strategically utilizing power-hungry fog servers only when mathematically justified to rescue failing deadlines, the algorithm slashed energy consumption by up to 93% compared to greedy heuristics and halved the energy drain of baseline quantum models, proving its viability as a robust, scalable, and sustainable orchestrator for next-generation edge networks.

6.2 Future Scope

While SHybridQIGA demonstrates exceptional performance in simulated digital twin environments, future work should focus on deploying the algorithm within physical, real-world edge hardware (such as physical Roadside Units and distributed Raspberry Pi clus-

ters) to validate its real-time operational overhead. Additionally, integrating advanced machine learning models, such as Recurrent Neural Networks (RNNs), for predictive vehicular trajectory tracking could allow the system to proactively pre-allocate server resources before traffic bottlenecks even form. The framework could also be extended to incorporate strict security and privacy constraints, ensuring that computationally offloaded tasks are protected against edge-level cyber threats. Finally, transitioning this quantum-inspired mathematical model to execution on genuine quantum computing hardware could further accelerate convergence speeds, unlocking unprecedented real-time orchestration capabilities for massive-scale Smart City infrastructures.

References

- [1] M. Bhatia, S. K. Sood, and S. Kaur, “Quantum-based predictive fog scheduler for iot applications,” *Computers in Industry*, vol. 111, pp. 51–67, 2019.
- [2] Qadri *et al.*, “Optimizing traffic signal timing at urban intersections: A simheuris-
tic approach using ga and sumo,” *IEEE Transactions on Intelligent Transportation
Systems*, 2024.
- [3] A. Schaffland *et al.*, “Simulating traffic networks: Driving sumo towards digital
twins,” in *SUMO User Conference 2024 Proceedings*, vol. 5. DLR, 2024, pp. 113–125.
- [4] Wu *et al.*, “Challenges and opportunities in fog computing scheduling: A literature
review,” *IEEE Access*, 2025.
- [5] M. H. Husain, M. Ahmadi, and F. Mardukhi, “A gwo-mfo-based resource allocation
in vehicular fog computing with latency constraints and energy reduction,” *Zanco
Journal of Pure and Applied Sciences*, vol. 37, no. 6, 2025.
- [6] Ahanger *et al.*, “Quantum-inspired task optimization algorithm specifically for iot
edge-fog computing environments,” *Future Generation Computer Systems*, 2023.
- [7] S. Galavani, A. Younesi, and M. Ansari, “Qiga: Quantum-inspired genetic algorithm
for dynamic scheduling in mobile edge computing,” in *2025 29th International Com-
puter Conference, Computer Society of Iran (CSICC)*. IEEE, 2025.
- [8] Anonymous *et al.*, “Qhrmof: A quantum-inspired hybrid multi-objective framework
for energy-efficient task scheduling and load balancing in cloud computing,” *Journal
of Cloud Computing*, vol. 14, no. 1, 2025.
- [9] M. Bey, P. Kuila, and B. B. Naik, “Quantum-inspired differential evolution with
decoding using hashing for efficient user allocation in edge computing environment,”
IEEE Transactions on Emerging Topics in Computing, 2024.

- [10] S. Madasamy, R. Vikkram, A. B. Reddy, T. Nandhini, S. Gupta, and A. Nagamani, “Predictive eqci-optimized load scheduling for heterogeneous iot-data in fog computing environments,” *IEEE Internet of Things Journal*, 2024.
- [11] Raj and Edwin, “Real-time vehicle trajectory and state prediction using advanced recurrent networks in a sumo simulated environment,” *Expert Systems with Applications*, 2025.

Appendix A: Presentation

FOG SCHEDULER USING QUANTUM PRINCIPLES

Guide: Dr. Jisha G.

Adithyan Ajayan U2203018

Ajay Mukund A U2203020

Alex Babu U2203030

Anna George U2203049

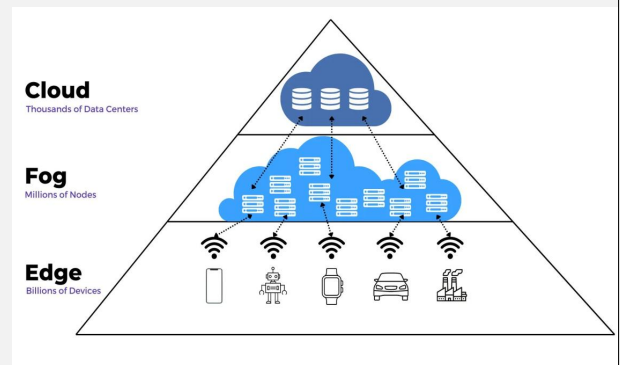
Bennette A Sabu U220306

CONTENTS

- Problem Definition
- Purpose & Need
- Project Objective
- Literature Survey
- Proposed Method
- Architecture Diagram
- Sequence Diagram
- Module Descriptions
- Modules in Detail
- Results & Outputs
- Assumptions
- Work Division
- Hardware & Software Requirements
- Gantt Chart
- Risks & Challenges
- Conclusion
- References

INTRODUCTION

- **Fog Computing** processes data at the network edge to minimize latency and improve response times for real-time applications like smart traffic and healthcare.
- **The Challenge** is to manage and schedule numerous simultaneous tasks remains a bottleneck for traditional methods, which struggle to adapt to dynamic workloads.
- By applying **Quantum-Inspired logic**, our system explores multiple scheduling solutions simultaneously. This avoids the "local optima" trap of traditional algorithms.

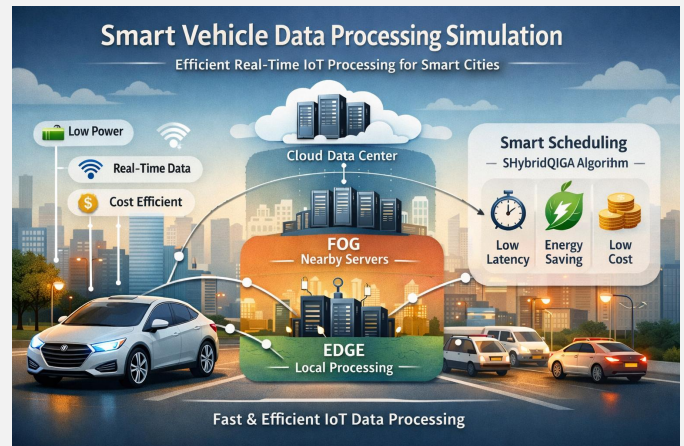


PROBLEM DEFINITION

To design a smart fog task scheduling system that reduces waiting time, keeps tasks balanced across fog nodes, and adapts quickly to changing workloads by using ideas from quantum computing (Quantum-Inspired Genetic Algorithms)

PURPOSE & NEED

- This project simulates a real-time data processing system for smart vehicles, helping to handle time-sensitive IoT data efficiently without relying on costly cloud processing.
- Instead of sending all data to the cloud, it uses a 3-tier architecture (Edge, Fog, Cloud). A Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA) schedules tasks efficiently, balancing latency, energy use, and cost.
- This enables faster, more sustainable data processing while reducing energy consumption and network expenses.



PROJECT OBJECTIVES

PHASE I

To design a simulation environment that optimizes task offloading in a heterogeneous 3-Tier network.

- **Develop a 3 – Tier Network:** Create a virtual model of Edge, Fog, and Cloud layers to simulate realistic network behaviour.
- **Multi-Objective Optimization:** Simultaneously minimize Latency (response time), Energy Consumption (power usage), and Monetary Cost (server cost).
- **Algorithm Benchmarking:** Compare Quantum-Inspired algorithms (QIGA, SHybridQIGA) against standard evolutionary methods (GA, PSO, NSGA-II).
- **Heterogeneous Environment:** Evaluate performance across diverse scenarios by varying task attributes.

PROJECT OBJECTIVES

PHASE 2

To apply the scheduling algorithm in a real-world-like setting where vehicles are constantly moving and network conditions keep changing.

- **System Integration:** Connect the traffic simulator (SUMO) with the network simulator to create a realistic smart city environment.
- **Spatial-to-Network Mapping:** Convert vehicle positions into network locations so the system can track movement and handle base station switching.
- **Heterogeneous Workloads:** Simulate different types of tasks from vehicles (like sensor data and video processing), especially during traffic congestion.
- **Performance Validation** Execute the algorithm continuously to monitor energy drain and minimize SLA violations during unpredictable traffic spikes.

LITERATURE SURVEY

Paper Title	Approach / Methodology	Strengths	Limitations
Optimizing Traffic Signal Timing at Urban Intersections: A Simheuristic Approach Using GA and SUMO	GA integrated with SUMO	Effectively links meta-heuristics with dynamic physical traffic via TraCI.	Focused strictly on traffic signals; does not address computational edge offloading.
Challenges and Opportunities in Fog Computing Scheduling: A Literature Review	Extensive Literature Review	Provides a clear taxonomy of fog constraints (energy, security, latency).	Identifies heuristic shortcomings but does not propose a novel architectural solution.
Quantum-Inspired Task Optimization algorithm specifically for IoT Edge-Fog computing environments	Quantum-Inspired Task Optimization	Faster convergence and better energy efficiency than classical GA.	Evaluated in static environments; ignores dynamic vehicular mobility and queuing bottlenecks.
Predictive EQCi-Optimized Load Scheduling for Heterogeneous IoT-Data in Fog Computing Environments	Predictive EQCi-Optimized Scheduling	Strong predictive capability using Node Availability Index; validated in iFogSim.	Assumes constant server processing speeds; lacks multi-core physical simulation scaling.
Real-Time Vehicle Trajectory and State Prediction using Advanced Recurrent Networks in a SUMO Simulated Environment	RNNs (GRU/BiLSTM) with SUMO	High accuracy in real-time trajectory and state prediction.	Lacks a scheduling or resource allocation mechanism for edge nodes.

LIMITATIONS OF STANDARD QIGA AND OUR PROPOSED ENHANCEMENT

- **Limitations of Standard QIGA**
 - **Random Initialization** → Many infeasible task-server mappings
 - **Poor Constraint Handling** → Deadline & memory violations
 - **Slow Convergence** → Not ideal for real-time fog environments
- **Why SHybridQIGA ?**
 - **Intelligent Safe Seeding** → Starts with a feasible, load-balanced solution
 - **Task-Level Repair Mechanism** → Fixes only invalid allocations
 - **Faster & Constraint-Aware Optimization** → Better for real-time offloading

DESIGN AND DEVELOPMENT OF THE PROPOSED SHQIGA

- **Quantum Probabilistic Initialization with Intelligent Safe Seeding**
 - Uses qubits to represent probabilistic task-server assignments. Instead of random start, a **load-balanced seeded schedule(Round Robin)** is introduced to ensure feasibility and faster convergence.
- **Quantum Representation & Measurement**
 - Quantum states are probabilistic, so they are **collapsed into binary values (0/1)** using measurement. This converts probabilistic solutions into **actual task-server mappings** for fitness evaluation.

DESIGN AND DEVELOPMENT OF THE PROPOSED SHQIGA

Energy-Aware Smart Repair Mechanism

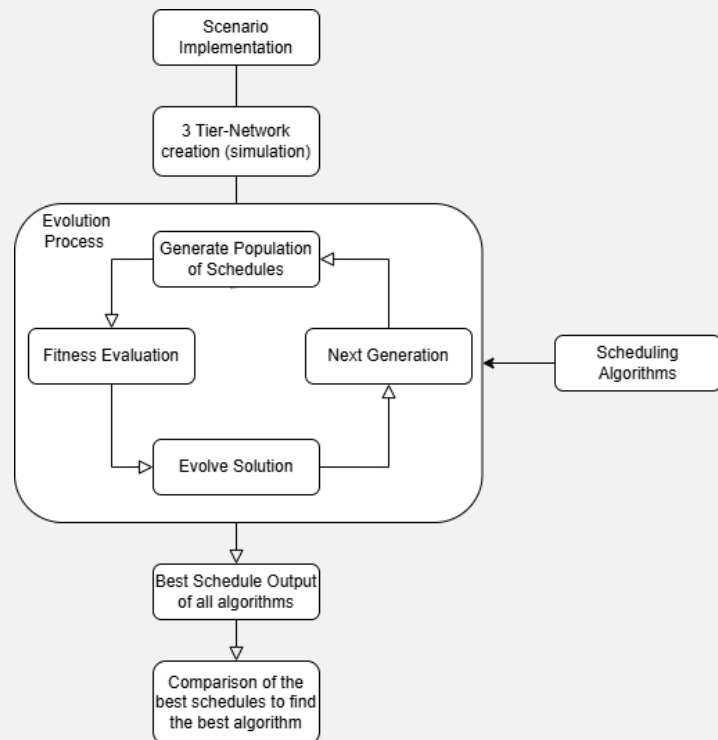
- Instead of penalizing the entire schedule, only violating task mappings are repaired.
Ensures deadline satisfaction, memory availability, and selects the **most energy-efficient valid server**.

Quantum Probabilities Update

- Qubits are updated using an **adaptive rotation gate**.
Starts with higher exploration and gradually shifts to fine-tuning, ensuring **steady and stable convergence**.

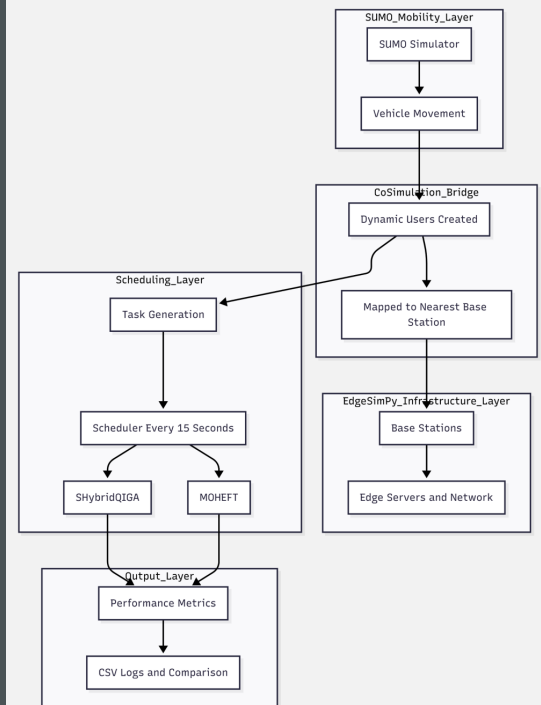
SYSTEM ARCHITECTURE PHASE I

- **Load Scenario:** The system reads a JSON dataset containing tasks and network layout.
- **Select Algorithm:** The system runs all algorithms on each scenario for a specific number of times to solve the problem.
- **Generate Schedules:** The algorithm proposes potential task-to-server assignments.
- **Evaluate Physics:** The Fitness Module calculates the Cost, Energy, and Latency for these assignments, with respect to deadline and memory constraints.
- **Evolve Solution:** The evolutionary algorithms use this feedback to "breed" better schedules over several generations.
- **Output Result:** Each algorithm provides with its best schedule. Based on the fitness values of these best schedules, we choose the best algorithm.

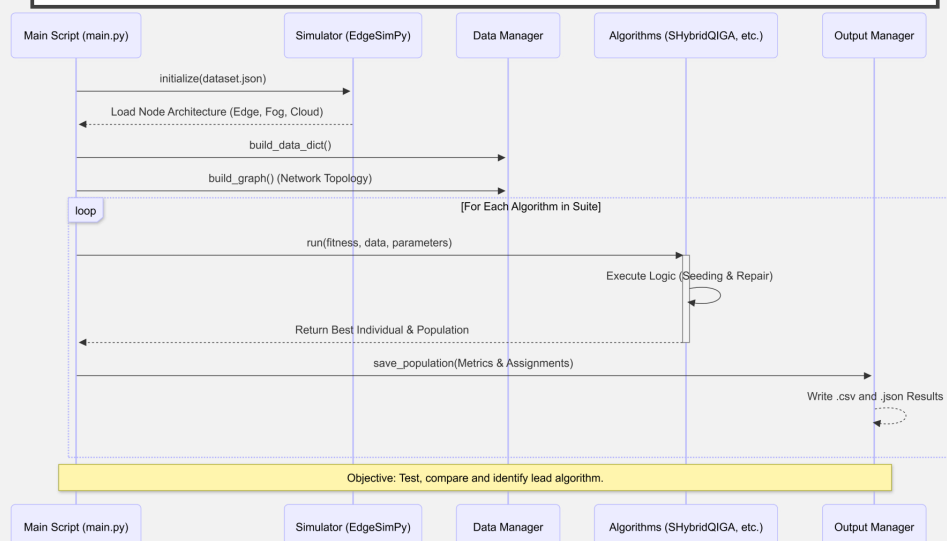


SYSTEM ARCHITECTURE PHASE 2

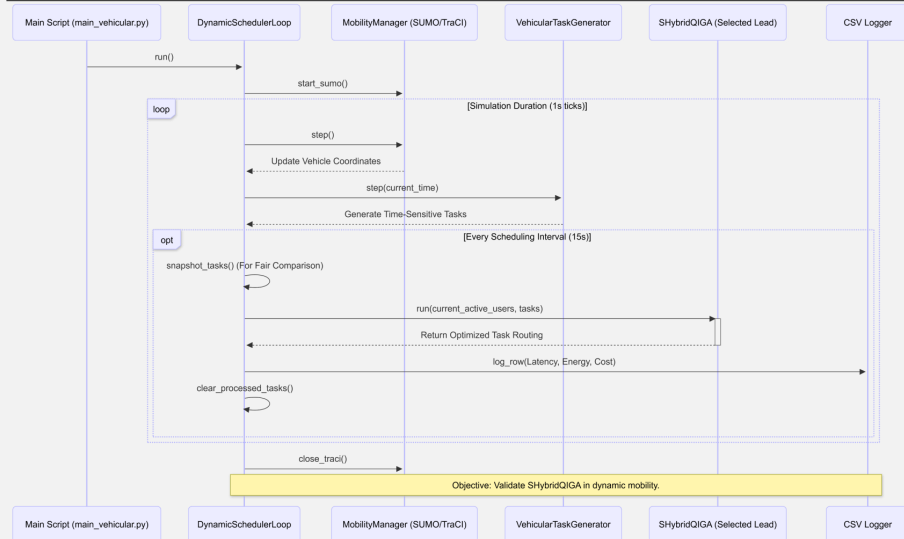
- **SUMO Traffic Simulation:** Uses the **SUMO** engine to simulate real-world vehicle movements based on `map.net.xml` and `routes.rou.xml`.
- **Mobility & Task Management:** A **Mobility Manager** tracks vehicle positions via TraCI, while a **Task Generator** creates periodic data requests from them.
- **3-Tier Infrastructure:** Built with **EdgeSimPy**, utilizing a network hierarchy of **Edge**, **Fog**, and **Cloud** layers.
- **Decision Engine (SHybridQIGA):** The **SHybridQIGA** algorithm optimizes task routing to specifically minimize server delay and energy consumption.
- **Monitoring & Visualization:** A **CSV Logger** tracks performance metrics like latency, power, SLA Violation Rate, etc. which are visualized on a **Streamlit Dashboard**.



SEQUENCE DIAGRAM PHASE I



SEQUENCE DIAGRAM PHASE 2



MODULE DESCRIPTION

- **Module 1: Edge-Fog-Cloud Infrastructure**

Engineers the backend network using **EdgeSimPy**, establishing the three-tier architecture and implementing precise power consumption calculations for all server layers.

- **Module 2: SHybridQIGA Optimization Engine**

Develops the **Smart Hybrid Quantum-Inspired Genetic Algorithm**, featuring intelligent seeding and repair mechanisms to optimize task routing for maximum energy and time efficiency.

- **Module 3: SUMO Traffic Mobility Integration**

Integrates the **SUMO simulator** to generate dynamic, real-time vehicle data, ensuring the network handles a continuous and realistic workload from active traffic.

MODULE DESCRIPTION

- **Module 4: Performance Visualization Dashboard**

Processes simulation output files to create an **interactive dashboard**, visualizing key metrics such as server delay, energy usage, and physical traffic waiting times.

- **Module 5: Benchmarking & Comparative Analysis**

Orchestrates rigorous simulation tests to evaluate performance, benchmarking the **SHybridQIGA** against standard models like **Round Robin** and traditional **Genetic Algorithms**.

MODULES IN DETAIL

MODULE 1: MULTI-TIER NETWORK INFRASTRUCTURE

- Developed using the **EdgeSimPy** library. It defines the hierarchical communication layers—**Edge, Fog, and Cloud**—and implements the mathematical models to calculate server power consumption.
- **Tier 1 Edge - Raspberry Pi 4** Base stations at data-producing regions. Ultra-low latency, highly energy-efficient with $P_{max} = 7.5W$, but heavily compute-constrained.
- **Tier 2 Fog - Intel Xeon E5430** Regional aggregation points offering a balanced middle-ground with $P_{max} = 200W$.
- **Tier 3 Cloud - 64-Core Server** Near-infinite processing capacity, but features a massive energy footprint with $P_{max} = 600W$, incentivizing local processing.

MODULE 1: MULTI-TIER NETWORK INFRASTRUCTURE

- **Execution Time Formula**

$$T_{exec} = \text{Task Workload (Cycles)} / \text{Server CPU Frequency(Hz)}$$

- **Network Delay Formula**

$$T_{path} = \frac{\text{Task Data Size}}{\text{Path Bandwidth}} + \text{Propagation Delay}$$

- **Total Time Taken**

$$T_{total} = T_{exec} + T_{path}$$

- **Dynamic CPU Utilization (U)**

$$U = \text{Task CPU Demand} / \text{Server Total CPU Capacity}$$

- **Linear Power Curve Formula**

$$P_{active} = P_{idle} + (P_{max} - P_{idle}) * U$$

- **Total Energy Formula**

$$E_{total} = P_{active} * T_{exec}$$

MODULE 1: MULTI-TIER NETWORK INFRASTRUCTURE

- **Monetary Cost (Ctotal)** Calculated as Server Rate multiplied by T_{exec} , financially penalizing the use of premium Tier 3 Cloud resources.
- **Min-Max Normalization** Converts Latency (Seconds), Energy (Joules), and Cost (Dollars) into a unified 0.0 to 1.0 scale to allow mathematical addition.
- **The Final Scalarization Formula (Fitness Score)**
$$F(x) = (w_{lat} * \hat{L}) + (w_{eng} * \hat{E}) + (w_{cost} * \hat{C}) + Penalty$$
- Weights are rigorously tuned: $w_{lat} = 0.7$, $w_{eng} = 0.2$, $w_{cost} = 0.1$.

MODULE 2: SHYBRIDQIGA OPTIMIZATION ENGINE

- This module focuses on the development of the **Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA)**. It encompasses the engineering of intelligent seeding and repair mechanisms designed to optimize task routing. The primary goal of this module is to minimize execution time and energy consumption through advanced quantum-inspired heuristics.

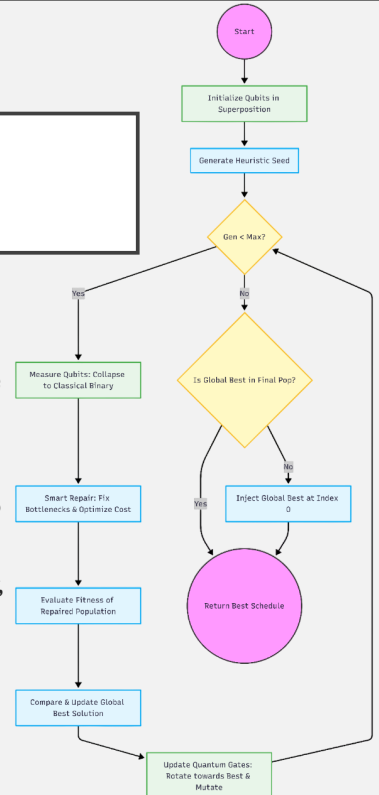
MODULE 2: SHYBRIDQIGA OPTIMIZATION ENGINE

- **Smart Initialization:** Start by distributing tasks across the best servers evenly.
- **Single Score Optimization:** Evaluate each solution using a combined score
- **Network vs Compute Check:** Compares execution delay and network delay; if network delay is higher, send to local base station, otherwise apply energy-efficient repair.
- **Energy-Efficient Repair:** Reassign tasks to low-power servers that can still meet deadlines.
- **Targeted Fixing:** Fix only the task assignments that violate deadlines.

MODULE 2: SHYBRIDQIGA OPTIMIZATION ENGINE

Algorithm Lifecycle Per Generation

- **Initialize and Seed:** Generate superposition qubits and inject the Safe Seed.
- **Measure:** Collapse probabilities into classical binary schedules.
- **Smart Repair:** Diagnose bottlenecks and apply Green Repair to failing tasks.
- **Evaluate Fitness:** Score schedules using normalized latency, energy, and cost.
- **Update Quantum Gates:** Apply the Rotation Gate to steer all qubits toward the Generation Champion.



MODULE 3: TRAFFIC MOBILITY & DATA GENERATION

- This module manages the environmental variables of the simulation using the **SUMO (Simulation of Urban MObility)** framework. It is responsible for integrating real-time moving vehicle data into the network. By simulating active traffic, this module ensures a continuous stream of data, creating a dynamic and realistic workload for the processing layers to handle.

MODULE 3: TRAFFIC MOBILITY & DATA GENERATION

- **Simulation Sync:** Utilizes the TraCI Traffic Control Interface API to keep traffic simulation and network simulation running in real time together.
- **Vehicle Tracking:** Continuously tracks each vehicle's position and movement.
- **Data Extraction:** Gets live data like vehicle ID, speed, and location.
- **Position Mapping:** Maps vehicle locations to the network grid.
- **Auto Connection:** Connects vehicles to the nearest base station.
- **Dynamic Handover:** Switches connection as vehicles move.
- **Vehicles as Nodes:** Treats each vehicle as a source of data tasks.

MODULE 3: TRAFFIC MOBILITY & DATA GENERATION

The Integrated Execution Cycle

- **Step 1 Advance Physics** Progresses the SUMO urban simulation forward by one physical time-step.
- **Step 2 Update State** Maps new vehicle coordinates and executes necessary Base Station handovers.
- **Step 3 Inject Workload** Vehicles dynamically generate new tasks.
- **Step 4 Scheduler Trigger** At fixed time intervals, the scheduler is invoked to handle latency-critical task scheduling.

MODULE 4: VISUALIZATION & ANALYTICS DASHBOARD

- This module focuses on the interpretability of simulation results. It involves the creation of an interactive dashboard and automated scripts to process raw output files. Key performance indicators (KPIs) such as energy usage, server latency, and vehicle waiting times are transformed into visual graphs to provide actionable insights into the system's behavior.

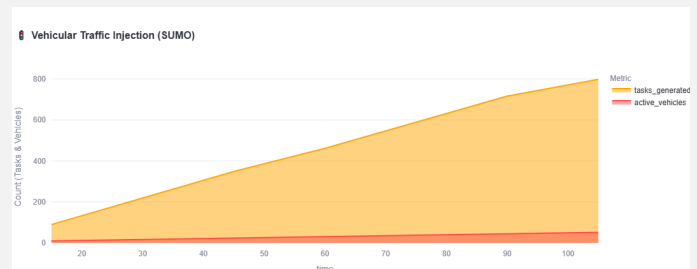
MODULE 4: VISUALIZATION & ANALYTICS DASHBOARD

- **Mathematical Normalization:** Converts different units (Latency in seconds, Energy in Joules, Cost in dollars) into a standardized 0.0 to 1.0 scale.
- **The Weighted Score (Phase I)** Combines these normalized metrics into a single score for comparing the algorithms, prioritizing latency($w=0.7$) most, then energy($w=0.2$) and cost($w=0.1$).

MODULE 4: VISUALIZATION & ANALYTICS DASHBOARD

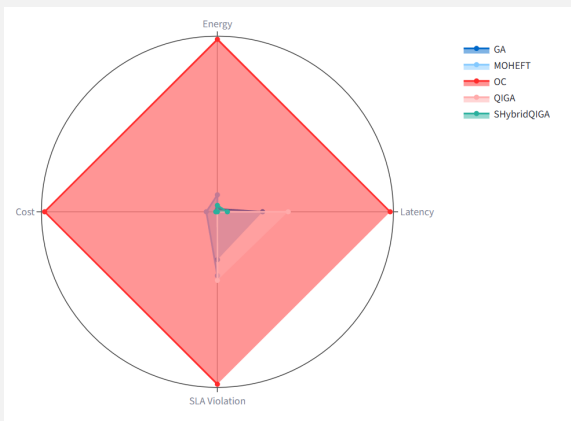


SLA Metric: Measures reliability using missed deadlines vs total tasks.

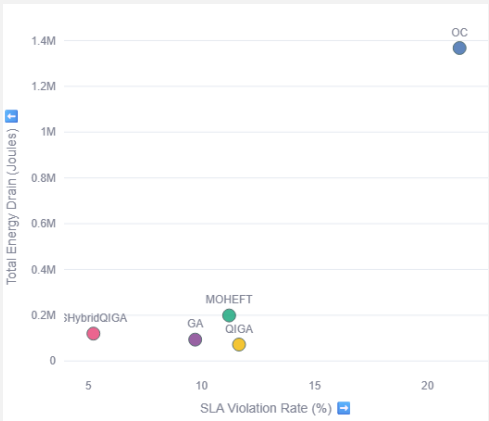


Stress vs Response The filled background area shows the volume of traffic being injected (the stress). The line charts show the algorithm's latency (the response).

MODULE 4: VISUALIZATION & ANALYTICS DASHBOARD



Radar Chart: Shows balance between latency, energy, cost, and SLA.



Pareto Plot: Highlights the trade-off between energy savings and missed deadlines.

MODULE 4: VISUALIZATION & ANALYTICS DASHBOARD

- **Auto Insights:** Automatically analyzes results and displays clear, human-readable performance improvements without needing to interpret graphs, for better algorithm comparison.

Algorithmic Impact & Trade-off Analysis

Use this section during your defense to prove Pareto-efficiency and quantify algorithmic improvements.

Dynamic Analytical Engine

Target Algorithm:

SHybridQIGA

Baseline Algorithm:

QIGA

Conclusion for Evaluator:

Comparing the metrics, SHybridQIGA reduced SLA Violations by 48.3% and improved average latency by 15.5% compared to the QIGA baseline.

Strategic Trade-off: To achieve its superior Quality of Service (QoS), SHybridQIGA strategically expended 54.4% more energy than QIGA by actively clearing network bottlenecks.

MODULE 5: EVALUATION & COMPARATIVE BENCHMARKING

- The final module is dedicated to rigorous testing and performance validation. It involves orchestrating simulation runs to evaluate the efficiency of the SHybridQIGA. This module provides a comparative analysis, benchmarking the custom algorithm against standard industry models like **Round Robin (RR)** and traditional **Genetic Algorithms (GA)** to quantify improvements in speed and resource management.

MODULE 5: EVALUATION & COMPARATIVE BENCHMARKING

The algorithms are stress-tested under different realistic conditions to see where they fail or succeed.

- Base Case:** A standard setup with balanced tasks and resources to provide a starting benchmark.
- Heavy Traffic:** A difficult scenario with many tasks but small number of nodes, testing if algorithms can handle "resource starvation".
- Heavy Data:** Focuses on tasks with large file sizes, which creates bottlenecks in network bandwidth and tests the communication efficiency.
- Heavy Computation:** Features tasks with massive CPU requirements, testing the processing power of the Edge and Fog servers.
- Critical:** Similar to Heavy Computation, but with a tight deadline.

Admin Controls

1. Generate New Scenario

Scenario Name

Base_Case

Number of Users

50

Tier 1 Nodes

15

Tier 2 Servers

4

Task Parameters:

Compute Weight (x 10e9)

3

Data Size (MB)

500

Deadline

30.00

Generate Dataset

MODULE 5: EVALUATION & COMPARATIVE BENCHMARKING

	Algorithm	↑ weighted_score
10	SHybridQIGA	11.6451
2	HybridQIGA	11.7209
3	MOHEFT	12.1515
7	QIGA	12.5168
0	DE	14.1487
9	RR	14.6091
11	SQIGA	14.6941
5	OE	14.7843
6	PSO	15.6397
1	GA	15.7643

BASE CASE

No. of tasks: 50
 No. of Tier 1 Nodes: 15
 No. of Tier 2 Servers: 4
 Compute Weight (Hz): 3×10^9
 Data Size (MB): 500

	Algorithm	↑ weighted_score
2	HybridQIGA	11.8161
9	RR	11.9973
7	QIGA	12.0094
5	OE	12.5038
0	DE	12.8681
8	RA	12.8723
11	SQIGA	12.9959
6	PSO	13.7369
10	SHybridQIGA	14.3829
1	GA	15.1372

HEAVY TRAFFIC

No. of tasks: 200
 No. of Tier 1 Nodes: 15
 No. of Tier 2 Servers: 4
 Compute Weight (Hz): 2×10^9
 Data Size (MB): 100

	Algorithm	↑ weighted_score
3	MOHEFT	37.8508
10	SHybridQIGA	41.5942
2	HybridQIGA	41.6713
1	GA	43.9439
11	SQIGA	46.9927
0	DE	47.4158
7	QIGA	47.5177
6	PSO	48.7277
5	OE	50.707
8	RA	51.4984

HEAVY DATA

No. of tasks: 50
 No. of Tier 1 Nodes: 15
 No. of Tier 2 Servers: 4
 Compute Weight (Hz): 2×10^9
 Data Size (MB): 4000

MODULE 5: EVALUATION & COMPARATIVE BENCHMARKING

	Algorithm	↑ weighted_score
10	SHybridQIGA	11.7663
2	HybridQIGA	14.4296
7	QIGA	15.4128
5	OE	20.0059
9	RR	20.5842
8	RA	22.9597
11	SQIGA	23.3304
0	DE	25.1273
6	PSO	26.0118
1	GA	41.3941

HEAVY COMPUTATION

No. of tasks: 50
 No. of Tier 1 Nodes: 15
 No. of Tier 2 Servers: 4
 Compute Weight (Hz): 2×10^9
 Data Size (MB): 4000

	Algorithm	↑ weighted_score
10	SHybridQIGA	11.1304
2	HybridQIGA	14.6049
7	QIGA	16.338
5	OE	21.7616
9	RR	23.9621
8	RA	24.2531
11	SQIGA	25.4839
0	DE	26.3455
6	PSO	28.6902
1	GA	42.3993

CRITICAL

No. of tasks: 50
 No. of Tier 1 Nodes: 15
 No. of Tier 2 Servers: 8
 Compute Weight (Hz): 9×10^9
 Data Size (MB): 100

RESULTS & OUTPUTS

- In Phase 1, testing across five distinct scenarios, SHybridQIGA outperformed competing algorithms in 3/5 cases , delivering exceptional energy efficiency and competitive latency.
- In Phase 2, across ~3100–3200 tasks, SHybridQIGA achieved the fewest failures (150–200) compared to GA/QIGA (300–500) and MOHEFT/Cloud (up to 1500).
- It also delivered the lowest latency, ensuring faster and more reliable execution.
- While using slightly more energy than GA/QIGA, it remains far more efficient than MOHEFT/Cloud and keeps costs low.

RESULTS & OUTPUTS

🏆 Cumulative Performance Leaderboard								
	Algorithm	Total_Tasks	↑ Total_Missed_Deadlines	SLA_Violation_Rate (%)	Average_Latency	Avg_Energy_Per_Task (J)	Total_Energy_Consumed	Total_Financial_Cost
4	SHybridQIGA	3,180	166	5.22%	1.081 s	49.128 J	156,226.5 J	\$7,920.81
0	GA	3,180	394	12.39%	1.528 s	34.819 J	110,724.9 J	\$6,891.24
1	MOHEFT	3,180	403	12.67%	1.093 s	93.363 J	296,895.4 J	\$14,104.42
3	QIGA	3,180	489	15.38%	1.671 s	28.648 J	91,100.2 J	\$6,888.45
2	OC	3,180	691	21.73%	1.957 s	406.759 J	1,293,495.0 J	\$61,595.00

BaseCase
Tier 1 : 20
Tier 2 : 5

🏆 Cumulative Performance Leaderboard								
	Algorithm	Total_Tasks	↑ Total_Missed_Deadlines	SLA_Violation_Rate (%)	Average_Latency	Avg_Energy_Per_Task (J)	Total_Energy_Consumed	Total_Financial_Cost
4	SHybridQIGA	3,219	147	4.57%	0.908 s	32.357 J	104,157.8 J	\$5,479.21
0	GA	3,219	322	10.00%	1.199 s	16.872 J	54,309.8 J	\$4,616.12
1	MOHEFT	3,219	369	11.46%	0.910 s	57.916 J	186,433.1 J	\$9,325.39
3	QIGA	3,219	373	11.59%	1.285 s	14.257 J	45,892.0 J	\$4,752.05
2	OC	3,219	702	21.81%	1.921 s	421.518 J	1,356,867.2 J	\$64,612.72

EdgeHigh
Tier 1 : 35
Tier 2 : 5

RESULTS & OUTPUTS



Cumulative Performance Leaderboard

	Algorithm	Total_Tasks	↑ Total_Missed_Deadlines	SLA_Violation_Rate (%)	Average_Latency	Avg_Energy_Per_Task (J)	Total_Energy_Consumed	Total_Financial_Cost
4	SHybridQIGA	3,199	167	5.22%	0.996 s	37.434 J	119,751.6 J	\$5,882.62
0	GA	3,199	311	9.72%	1.211 s	29.177 J	93,336.2 J	\$5,337.68
1	MOHEFT	3,199	359	11.22%	0.935 s	62.169 J	198,878.2 J	\$9,174.61
3	QIGA	3,199	373	11.66%	1.368 s	22.473 J	71,892.1 J	\$5,369.50
2	OC	3,199	685	21.41%	1.992 s	427.431 J	1,367,351.4 J	\$65,111.97

ServerHigh

Tier 1 : 20

Tier 2 : 15



Cumulative Performance Leaderboard

	Algorithm	Total_Tasks	↑ Total_Missed_Deadlines	SLA_Violation_Rate (%)	Average_Latency	Avg_Energy_Per_Task (J)	Total_Energy_Consumed	Total_Financial_Cost
4	SHybridQIGA	3,200	134	4.19%	0.818 s	15.447 J	49,429.3 J	\$2,920.39
0	GA	3,200	216	6.75%	0.911 s	11.029 J	35,294.1 J	\$2,700.55
1	MOHEFT	3,200	252	7.88%	0.768 s	29.603 J	94,728.6 J	\$4,873.58
3	QIGA	3,200	259	8.09%	0.968 s	10.003 J	32,009.3 J	\$2,808.27
2	OC	3,200	670	20.94%	1.944 s	401.645 J	1,285,263.0 J	\$61,203.00

IncreasedServers

Tier 1 : 55

Tier 2 : 15

ASSUMPTIONS

- **Continuous Connectivity:** Vehicles stay connected to infrastructure at all times.
- **Fixed Infrastructure:** Edge/Fog nodes are static; only vehicles move.
- **Task Homogeneity:** Vehicular tasks follow predictable data generation patterns as defined in the simulation parameters.
- **Simplified Energy Model:** Uses standard power consumption models.
- **Predefined Resources:** Bandwidth and memory are fixed and available.

WORK DIVISION

- **Ajay Mukund A:** Designed the core simulation framework and developed the custom **Smart Hybrid Quantum-Inspired Genetic Algorithm (SHybridQIGA)**, engineering novel intelligent seeding and repair mechanisms for optimized task routing to save time and energy.
- **Adithyan Ajayan:** Integrated the **SUMO traffic simulator** to introduce real moving cars into the project, ensuring the network handles continuous data generation from active vehicles.
- **Anna George:** Engineered the backend network simulation using the **EdgeSimPy library**, creating the **Edge, Fog, and Cloud layers** and developing the logic to calculate precise server power consumption.

WORK DIVISION

- **Alex Babu:** Built the **interactive dashboard and data graphs**, writing the scripts to process output files and clearly display metrics like energy usage, server delay, and physical traffic waiting times.
- **Bennette A Sabu:** Orchestrated the **simulation tests and performance analysis**, evaluating efficiency and speed improvements by benchmarking the new algorithm against standard models like **Round Robin** and **Genetic Algorithms**.

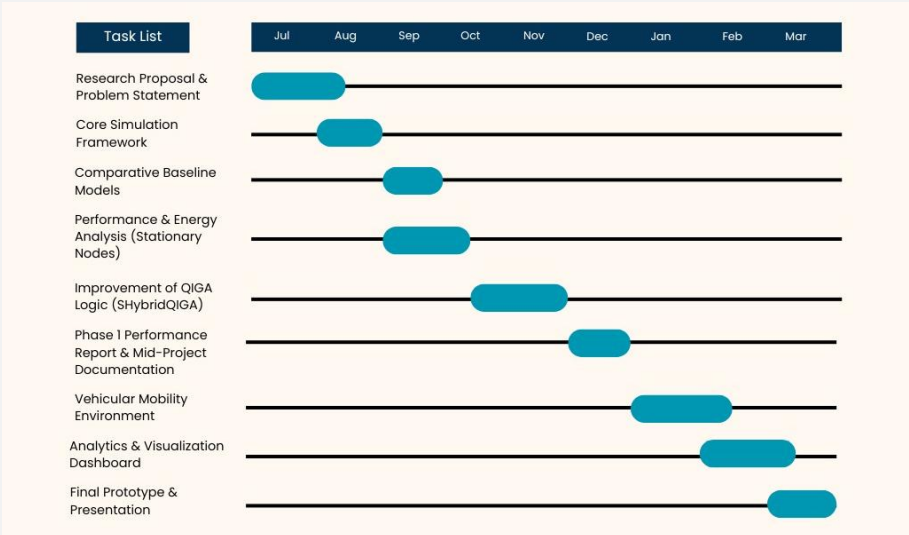
HARDWARE & SOFTWARE REQUIREMENTS

- Software Requirements
 - **Programming Language:** Python 3.9+
 - **Traffic Simulation:** Eclipse SUMO (Simulation of Urban MObility)
 - **Network Modeling:** EdgeSimPy Library
 - **Data Visualization:** Streamlit (Dashboard), Plotly (Analytics), and PyVis (Network Topology)
 - **Integration:** TraCI (Traffic Control Interface) for real-time Python-SUMO communication

HARDWARE & SOFTWARE REQUIREMENTS

- Hardware Requirements
 - **Processor:** Multi-core CPU (Intel i5/i7 or equivalent) to handle parallel genetic algorithm iterations and physics simulations.
 - **Memory:** 16 GB RAM (Recommended) for simultaneous execution of SUMO, the backend network simulation, and interactive dashboards.
 - **Graphics:** Dedicated GPU recommended for physics-enabled PyVis network rendering and large-scale data visualization.
 - **Storage:** High-speed SSD for rapid processing of simulation logs and output files.

GANTT CHART



RISKS & CHALLENGES

- **Mobility Handovers:** Maintaining connectivity as fast-moving vehicles transition between network nodes.
- **Computational Latency:** Ensuring complex quantum-inspired logic executes within strict real-time windows.
- **Scalability Limits:** Managing massive vehicle density without server memory or simulation bottlenecks.
- **Reality Gap:** Transitioning from software simulation to unpredictable real-world signal interference.
- **Conflict Resolution:** Balancing the strictly opposing goals of energy, cost, and latency.

CONCLUSION

Our project evolved in two key phases to build a high-performance vehicular computing framework.

In **Phase 1**, we developed the 3-tier architecture (Edge, Fog, Cloud) and focused on algorithm design and testing. SHybridQIGA emerged as the best scheduler, offering an optimal balance between speed and energy efficiency.

In **Phase 2**, we integrated the SUMO simulator to test the system in a dynamic, real-world environment with moving vehicles and continuous data. The results confirmed the algorithm's robustness and reliability.

Ultimately, the project delivers a scalable solution that makes vehicular networks faster, more efficient, and reliable.

REFERENCES

- **[1]** Qadri et al., "Optimizing Traffic Signal Timing at Urban Intersections: A Simheuristic Approach Using GA and SUMO," *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- **[2]** Wu et al., "Challenges and Opportunities in Fog Computing Scheduling: A Literature Review," *IEEE Access*, 2025.
- **[3]** Ahanger et al., "Quantum-Inspired Task Optimization Algorithm for IoT Edge-Fog Computing Environments," *Future Generation Computer Systems*, 2023.
- **[4]** S. Madasamy, R. Vikkram, A. B. Reddy, T. Nandhini, S. Gupta, and A. Nagamani, "Predictive EQCi-Optimized Load Scheduling for Heterogeneous IoT Data in Fog Computing Environments," *IEEE Internet of Things Journal*, 2024.
- **[5]** Raj and Edwin, "Real-Time Vehicle Trajectory and State Prediction Using Advanced Recurrent Networks in a SUMO Simulated Environment," *Expert Systems with Applications*, 2025.
- **[6]** M. H. Husain, M. Ahmadi, and F. Mardukhi, "A GWO-MFO-Based Resource Allocation in Vehicular Fog Computing with Latency Constraints and Energy Reduction," *Zanco Journal of Pure and Applied Sciences*, vol. 37, no. 6, 2025.

REFERENCES

- **[7]** S. Galavani, A. Younesi, and M. Ansari, "QIGA: Quantum-Inspired Genetic Algorithm for Dynamic Scheduling in Mobile Edge Computing," in *Proc. 29th Int. Computer Conf. Computer Society of Iran (CSICC)*, 2025, doi:10.1109/CSICC65765.2025.10967435.
- **[8]** A. Schaffland et al., "Simulating Traffic Networks: Driving SUMO Towards Digital Twins," in *SUMO User Conference 2024 Proceedings*, vol. 5, pp. 113–125, 2024.
- **[9]** M. Bey, P. Kuila, and B. B. Naik, "Quantum-Inspired Differential Evolution with Decoding Using Hashing for Efficient User Allocation in Edge Computing Environment," *IEEE Transactions on Emerging Topics in Computing*, 2024.
- **[10]** Anonymous et al., "QHRMOF: A Quantum-Inspired Hybrid Multi-Objective Framework for Energy-Efficient Task Scheduling and Load Balancing in Cloud Computing," *Journal of Cloud Computing*, vol. 14, no. 1, 2025.
- **[11]** M. Bhatia, S. K. Sood, and S. Kaur, "Quantum-Based Predictive Fog Scheduler for IoT Applications," *Computers in Industry*, vol. 111, pp. 51–67, 2019, doi:10.1016/j.compind.2019.06.001.

THANK YOU

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to

one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills: The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills: The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills: The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse

teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

Mapping Course Outcomes (COs) – Program Outcomes (POs) & Course Outcomes (COs) – Program Specific Outcomes (PSOs)

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
1	2	2	2	1		2	1	1	1	1	1	2	2	2
2	2	2	2		2	3	2	1	1	1	1	2	2	
3								3	2	2	1			3
4		1	2		3	2	3	2	2	3	1		2	2
5	2	3	3	1	2					1	3	3		
6			2			2	2	2	3	1	1			3

Justifications for Mapping

Mapping	Low / Medium / High	Justification
101003/ CS822U.CO1- PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.

Mapping	Low / Medium / High	Justification
101003/ CS822U.CO1- PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1- PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1- PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1- PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1- PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1- PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1- PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1- PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/ CS822U.CO1- PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/ CS822U.CO1- PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.
101003/ CS822U.CO1- PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1- PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2- PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.

Mapping	Low / Medium / High	Justification
101003/ CS822U.CO2- PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2- PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2- PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2- PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2- PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.
101003/ CS822U.CO2- PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2- PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2- PO10	Low	Requires basic planning and coordination during development tasks.
101003/ CS822U.CO2- PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/ CS822U.CO2- PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2- PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/ CS822U.CO3- PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3- PO9	Medium	Requires effective communication among team members during collaboration.

Mapping	Low / Medium / High	Justification
101003/ CS822U.CO3- PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3- PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/ CS822U.CO3- PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/ CS822U.CO4- PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4- PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4- PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4- PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4- PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/ CS822U.CO4- PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/ CS822U.CO4- PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4- PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/ CS822U.CO4- PO11	Low	Encourages adaptability and continuous learning during project implementation.
101003/ CS822U.CO4- PSO2	Medium	Uses software development practices to execute and manage project tasks.

Mapping	Low / Medium / High	Justification
101003/ CS822U.CO4- PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/ CS822U.CO5- PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5- PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5- PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5- PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5- PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5- PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/ CS822U.CO5- PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/ CS822U.CO5- PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.
101003/ CS822U.CO6- PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6- PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6- PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6- PO8	Medium	Communication supports collaboration within team environments.

Mapping	Low / Medium / High	Justification
101003/ CS822U.CO6- PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6- PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6- PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6- PSO3	High	Communication of innovative computer science solutions and research outcomes.